



MANUEL D'UTILISATION

Carte CAN PC/104

VERSION PRELIMINAIRE

Cette documentation est une version préliminaire.

Copyright © 1998 ATON Systèmes

IBM PC/AT® est une marque déposée d'International Business Machines Corporation.

MS/DOS® est une marque déposée de Microsoft Corporation.

PC/104 est une marque déposée du Consortium PC/104.

Le CAN est un protocole qui est enrichi par l'association CiA (CAN in Automation) regroupant plus de 250 compagnies.

Toute ou partie de cette documentation ne peut être reproduite ou traduite par un moyen quelconque, sans l'accord écrit de la société ATON Systèmes.

Ce document est fourni à titre d'information seulement. Il n'engage pas la responsabilité de la société ATON Systèmes en cas de dommages résultant de son application.

ATON Systèmes cherche continuellement à améliorer ses produits; des corrections ou modifications au contenu de ce document peuvent intervenir sans préavis.

www.aton-sys.fr
e-mail : support@aton-sys.fr

juin 1999

SOMMAIRE

PRESENTATION	5
CONFIGURATION DE LA CARTE CAN.....	8
PRESENTATION	8
ACCES I/O à la CARTE CAN	8
Configuration des adresses I/O	9
PRINCIPE d'ACCES au contrôleur CAN SJA1000.....	10
Gestion des Interruptions	11
LIAISON OPTOISOLEE.....	12
TOPOLOGIE DU RESEAU ET SUPPORT DE TRANSMISSION.....	14
Topologie du réseau	14
Choix du support de transmission.....	15
CONNECTEURS	17
Connecteur CAN:.....	17
Connecteurs "Standard CIA"	18
Connecteur PC/104	19
Carte CAN PC/104 : Connecteurs et cavaliers	22
INSTALLATION DES DRIVERS ET UTILITAIRES	26
MS-DOS	26
DRIVER	26
INSTALLATION AUTOMATIQUE	27
INSTALLATION MANUELLE.....	27
UTILITAIRES	28
CANTERM	28
Windows 3.1x	31
DRIVER	31
INSTALLATION AUTOMATIQUE	31
INSTALLATION MANUELLE.....	31
UTILITAIRES	32
Windows CAN Terminal (WCT).....	32
CAN Spy (CANSPY).....	34
Windows 95	36
DRIVER	36
INSTALLATION AUTOMATIQUE	37
INSTALLATION MANUELLE.....	37
UTILITAIRES	37
PROGRAMMATION	38

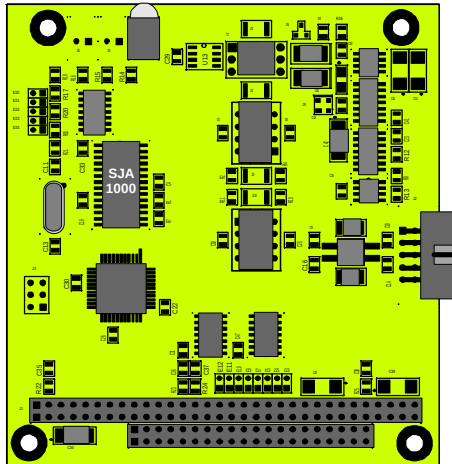
GENERALITES.....	38
Le CAN par ATON.....	38
Identification des messages	38
Filtrage des messages	40
Canaux CAN	40
Fonctionnement général des drivers et des applications.....	42
STRUCTURES ESSENTIELLES.....	44
Définition de la configuration matérielle	44
Messages ACAN.....	48
Filtres à messages	49
Tampon de communication (<i>Task Block</i>).....	50
INTERFACE DE PROGRAMMATION.....	56
ACAN_Init	57
ACAN_GetDriverVersion	57
ACAN_GetInitStatus	58
ACAN_SetHardConfig.....	58
ACAN_GetHardConfig.....	59
ACAN_BusControl	60
ACAN_Register	61
ACAN_AckEvent, ACAN_AckEventCheck	64
ACAN_Deregister.....	66
ACAN_SendMessage.....	66
ACAN_RetrieveMessages	67
ACAN_AllocFixedMemory.....	67
ACAN_FreeFixedMemory	68
ACAN_Exit	68
BIBLIOTHEQUE C/C++.....	70
MS-DOS.....	70
Windows 3.1x	70
Windows 95.....	71
ANNEXE A : Interface de programmation assembleur.....	72
Test de présence du driver ACAN.....	72
Appel des services du driver	73
FICHE DE RAPPORT D'INCIDENT.....	78

PRESENTATION

Ce manuel est consacré à la carte CAN PC/104 quelles que soient les options d'équipement. Il permet de configurer la carte pour son utilisation.

Description générale

La carte CAN PC/104 est développée autour du contrôleur CAN "Stand Alone" **SJA1000 Philips**. Associé à une carte microprocesseur compatible IBM/PC, PC/AT ou PC/104, elle permet rapidement la construction d'un réseau CAN pour des applications industrielles. Elle possède, par ailleurs, le driver de ligne nécessaire pour assurer la transmission de données en mode "High speed" (Jusqu'à 1 Mbauds). L'adresse de base de la carte ainsi que le niveau d'interruption sont configurables par cavaliers directement sur la carte. Dans sa version isolée, la carte CAN est équipée de deux optocoupleurs 6N137 au niveau de la transmission et la réception des données. Dans ce cas, un convertisseur DC/DC 5V/5V est présent pour assurer l'alimentation de la partie isolée.



Les drivers et bibliothèques contenus dans la disquette sont développés par **ATON Systèmes**. Les bibliothèques en langage assembleur ainsi qu'en langage C peuvent être utilisées pour le développement d'applications spécifiques.

L'ensemble des fonctions et des connecteurs de la carte CAN/PC104 est compatible IBM PC/AT ou IBM PS/2.

Composition

CONFIGURATION STANDARD:

Contrôleur SJA1000 Philips:	Supporte les trames CAN 2.0A et 2.0B Vitesse de transmission jusqu'à 1M/bauds.
CPLD XC9536 Xilinx:	Décodage d'adresse et accès au SJA1000
PCA82C250 Philips (ou 82C251):	Driver de ligne High speed
Témoins lumineux:	LED Jaune pour les interruptions

CONFIGURATION OPTOISOLEE:

Optocoupleurs:	6N137 à temps de propagation rapide
Convertisseur DC/DC:	Convertisseur 5V/5V 250mA

INTERFACES:

Connecteur PC/104
Connecteur HE_10 2x5 points

Spécifications

Electriques:

+5V $\pm 5\%$ @500 mA Maximum (Carte mono-tension)

Environnement:

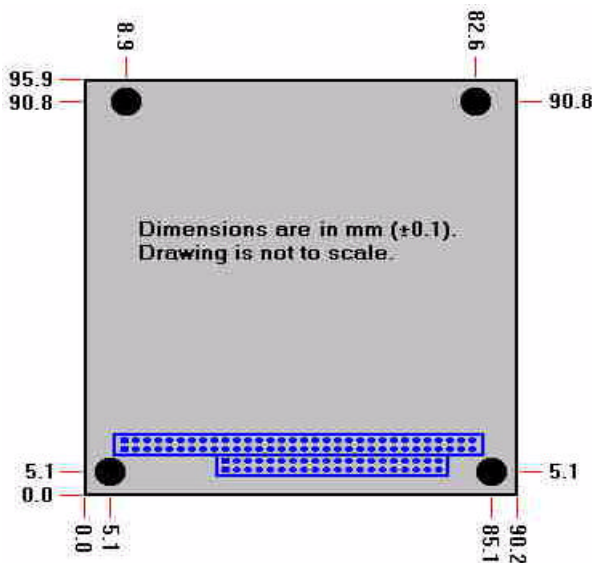
Température de fonctionnement : 0 / 70 °C

Température de stockage : -20 / 125 °C

Taux d'humidité : 10 à 90% sans condensation

Taux d'humidité : 5 à 95% sans condensation (stockage)

Dimensions:



CONFIGURATION DE LA CARTE CAN

PRESENTATION

Sur la carte, les cavaliers sont répartis par fonctions :

- Sélection de l'adresse de base : E20 à E24
- Sélection du niveau d'interruption: E3-E4-E5-E7-E9-E10-E11-E12

Note :

*** :** *Position par défaut*

P : *Présent*

NP : *Non présent*

Les positions des cavaliers sont décrites dans les paragraphes suivants.

ACCES I/O à la CARTE CAN

La configuration de l'adresse d'accès de la carte CANPC/104 s'effectue par le positionnement des cavaliers E20 à E24.

L'accès à la carte CAN se fait en I/O 8 bits (standard) depuis le bus ISA du connecteur PC/104. A partir d'une adresse de base déterminée par l'utilisateur, on peut accéder à toutes les ressources du contrôleur CAN.

Configuration des adresses I/O

Les 5 cavaliers permettent la configuration de 24 adresses de base différentes. Le décodage d'adresse a lieu dans l'espace compatible I/O du bus AT, c'est-à-dire de l'adresse **100h à 3FFh**.

Adresses de base:

E24 (S9)	E23 (S8)	E22 (S7)	E21 (S6)	E20 (S5)	Adresse
<i>P</i>	<i>NP</i>	<i>P</i>	<i>P</i>	<i>P</i>	100h
<i>P</i>	<i>NP</i>	<i>P</i>	<i>P</i>	<i>NP</i>	120h
<i>P</i>	<i>NP</i>	<i>P</i>	<i>NP</i>	<i>P</i>	140h
<i>P</i>	<i>NP</i>	<i>P</i>	<i>NP</i>	<i>NP</i>	160h
<i>P</i>	<i>NP</i>	<i>NP</i>	<i>P</i>	<i>P</i>	180h
<i>P</i>	<i>NP</i>	<i>NP</i>	<i>P</i>	<i>NP</i>	1A0h
<i>P</i>	<i>NP</i>	<i>NP</i>	<i>NP</i>	<i>P</i>	1C0h
<i>P</i>	<i>NP</i>	<i>NP</i>	<i>NP</i>	<i>NP</i>	1E0h
<i>NP</i>	<i>P</i>	<i>P</i>	<i>P</i>	<i>P</i>	200h
<i>NP</i>	<i>P</i>	<i>P</i>	<i>P</i>	<i>NP</i>	220h
<i>NP</i>	<i>P</i>	<i>P</i>	<i>NP</i>	<i>P</i>	240h
<i>NP</i>	<i>P</i>	<i>P</i>	<i>NP</i>	<i>NP</i>	260h
<i>NP*</i>	<i>P*</i>	<i>NP*</i>	<i>P*</i>	<i>P*</i>	280h
<i>NP</i>	<i>P</i>	<i>NP</i>	<i>P</i>	<i>NP</i>	2A0h
<i>NP</i>	<i>P</i>	<i>NP</i>	<i>NP</i>	<i>P</i>	2C0h
<i>NP</i>	<i>P</i>	<i>NP</i>	<i>NP</i>	<i>NP</i>	2E0h
<i>NP</i>	<i>NP</i>	<i>P</i>	<i>P</i>	<i>P</i>	300h
<i>NP</i>	<i>NP</i>	<i>P</i>	<i>P</i>	<i>NP</i>	320h
<i>NP</i>	<i>NP</i>	<i>P</i>	<i>NP</i>	<i>P</i>	340h
<i>NP</i>	<i>NP</i>	<i>P</i>	<i>NP</i>	<i>NP</i>	360h
<i>NP</i>	<i>NP</i>	<i>NP</i>	<i>P</i>	<i>P</i>	380h
<i>NP</i>	<i>NP</i>	<i>NP</i>	<i>P</i>	<i>NP</i>	3A0h
<i>NP</i>	<i>NP</i>	<i>NP</i>	<i>NP</i>	<i>P</i>	3C0h
<i>NP</i>	<i>NP</i>	<i>NP</i>	<i>NP</i>	<i>NP</i>	3E0h

Quel que soit le mode d'utilisation du contrôleur par le driver ACAN (BasicCAN ou PeliCAN) l'espace réservé sera toujours de 32 octets dans l'espace compatible AT.

PRINCIPE d'ACCES au contrôleur CAN SJA1000

Le circuit SJA1000 comporte 32 registres accessibles en mode **BasicCAN** et 128 en mode **PeliCAN**. Pour éviter de réserver trop d'espace mémoire dans la zone compatible, on utilise un mode d'accès par page pour atteindre la totalité des 128 octets. On utilise les bits 0 et 4 du bus d'adresse (A0-A11) pour les 32 octets et les bits 10 et 11 pour le reste de l'espace adressable.

Adresse Offset: Adresse A(0:4) pour l'accès à 32 octets
Adresse Page: Adresse A11 et A10 pour l'accès à la totalité de l'espace mémoire du SJA1000.

Pour exemple, si l'adresse de base est 0x280 :

Mode	Adresse	
BasicCan	0280h -> 029Fh	IO 8 bits In/Out 32 registres
PeliCan	0280h -> 029Fh	IO 8 bits In/Out Page 0 Reg. (0-32)
	0680h -> 069Fh	IO 8 bits In/Out Page 1 Reg. (33-64)
	0A80h -> 0A9Fh	IO 8 bits In/Out Page 2 Reg. (65-96)
	0E80h -> 0E9Fh	IO 8 bits In/Out Page 3 Reg. (97-128)

Les pages 1, 2 et 3 sont réservées en dehors de l'espace compatible AT.

L'utilisateur ne peut configurer que l'adresse de base de la carte. Il doit retenir que 32 octets sont retenus dans l'espace compatible AT et que 96 autres octets sont réservés dans les trois pages successives.

Gestion des Interruptions

Il y a 8 niveaux d'interruptions configurables sur la carte:

Niveaux d'interruption	Cavaliers
3	E3
4	E4
5*	E5
7	E7
9	E9
10	E10
11	E11

Le niveau d'interruption utilisé correspond au "shunt" présent sur le cavalier.

Une LED Jaune est le témoin du niveau de l'interruption:

- Eteinte: Pas d'interruption
- Allumée: Interruption

Remarque:

La carte CAN PC/104 n'est pas prévue pour fonctionner en "Interrupt Sharing"

LIAISON OPTOISOLEE

Dans le cas où la carte CAN est isolée (selon l'option demandée), les liaisons entre le driver de ligne et le contrôleur CAN SJA1000 sont isolées.

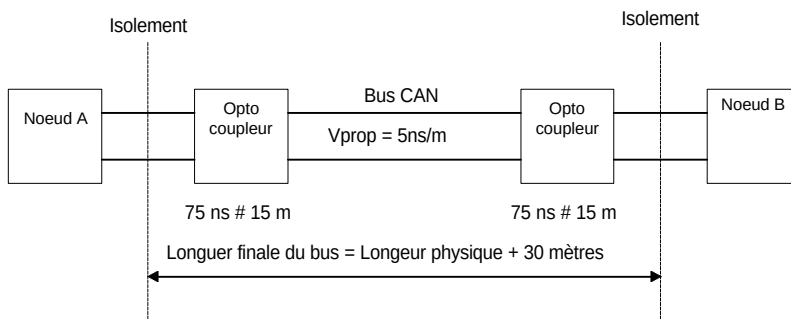
L'isolement est assuré grâce à un optocoupleur sur la ligne de transmission et un autre sur la ligne de réception. Les circuits utilisés sont des 6N137 dont les caractéristiques sont les suivantes:

- Tension d'isolement entre l'entrée et la sortie : 2500 Vrms
- Temps de passage Etat haut /Etat bas ou Etat bas/Etat haut de 45 ns (typique) à 75 ns (max)

Un convertisseur DC/DC 5V/5Vest présent sur la carte. Son utilisation est dédiée, exclusivement, à l'alimentation de la partie isolée (driver de ligne CAN et optocoupleurs).

L'utilisation d'optocoupleurs sur un réseau quel qu'il soit introduit un temps de propagation qui sera considéré comme une "augmentation virtuelle" du support de transmission.

Dans le cas de l'utilisation des 6N137, le temps de propagation est de 75 ns au maximum. Donc, pour un temps de propagation dans le médium de 5 ns/m, la longueur accumulée équivalente lors du passage dans l'optocoupleur sera de 15 mètres.



Le signal va être transmis et reçu en passant par deux optocoupleurs. Le retard accumulé est donc 150 ns ($2 \times 75 \text{ ns}$) soit 30 mètres de câble en plus.

Ce retard oblige à revoir à la baisse la vitesse de transmission ou la longueur du câble. C'est un compromis qui dépend de l'application.

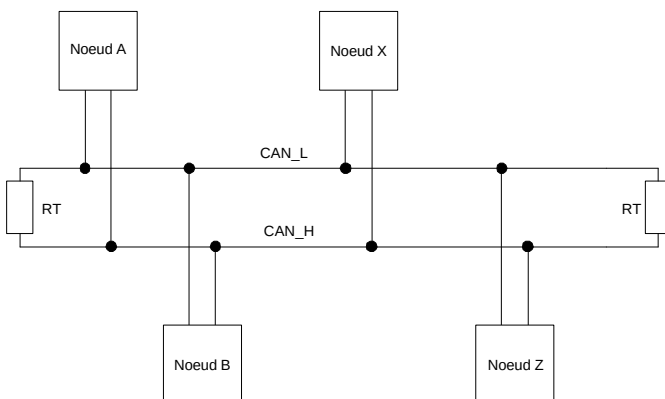
Remarque:

Il est conseillé d'utiliser une liaison optocouplée pour des distances de transmission supérieures à 200 mètres.

TOPOLOGIE DU RESEAU ET SUPPORT DE TRANSMISSION

Topologie du réseau

La topologie d'un réseau CAN est : **Le BUS**



Les résistances de terminaison RT doivent être placées aux extrémités du câble de transmission. La valeur de RT varie de 120 à 180 Ohms.

A 1Mbits/s les dérivations (*stubs*) ne doivent pas dépasser 30 cm.

Choix du support de transmission

On peut utiliser deux types de support: paire différentielle simple, paire différentielle torsadée (blindée ou non). Pour une application comme le réseau CAN, la paire torsadée est conseillée.

Longueur du médium

La longueur maximum du médium dépend essentiellement du retard accumulé au niveau de la transmission (temps de propagation et temps de traitement matériel). Le tableau suivant donne la longueur du câble en fonction de la vitesse de transmission.

Vitesse de propagation	Longueur du câble
1 Mbit/s	25 m
800 Kbit/s	50 m
500 Kbit/s	100 m
250 Kbit/s	250 m
125 Kbit/s	500 m

Un compromis doit être fait entre la longueur du médium et la vitesse de transmission.

Remarque:

Dans le cas d'une carte optoisolée, il faut tenir compte du retard accumulé dans les optocoupleurs...

Nombre de nœuds

Le nombre de nœuds sur le bus CAN est déterminé par la technologie du driver de ligne ainsi que par le type de câble utilisé. Ces deux éléments influent également sur la longueur du support de transmission. Le tableau suivant donne en fonction du câble et du nombre de nœuds la longueur du support de transmission:

Type de câble	n = 32	n = 64	n = 100
Thin cable	L = 200 m	L = 170 m	L = 150 m
Thick cable	L = 800 m	L = 690 m	L = 600 m
AWG 20/ 0.5 mm ²	L = 360 m	L = 310 m	L = 270 m
AWG 18/0.75 mm ²	L = 550 m	L = 470 m	L = 410 m

n = nombre de noeuds

L =longueur maximum du support de transmission

Le driver de ligne utilisé est capable de supporter jusqu'à 110 nœuds.

Remarque:

AWG est une norme. Elle correspond à la composition du câble utilisé: section, monobrin ou multibrins, résistance de ligne...

CONNECTEURS

Tous les connecteurs sont numérotés (Jx) et l'origine (le "1") est repérée par une pastille carrée sur le côté soudure du circuit imprimé. L'utilisation de connecteur type HE10 avec un nez de détrompage évitera les erreurs.

Connecteur CAN:

L'accès au réseau CAN se fait par le connecteur HE-10 "**J2**".

1 - N.C	6 - N.C
2 - Masse	7 - N.C
3 - CAN_L	8 - N.C
4 - CAN_H	9 - N.C
5 - Masse	10 - N.C

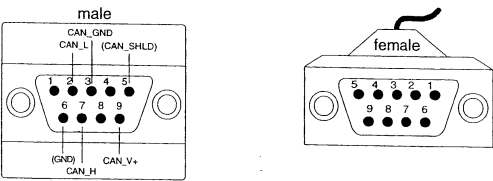
L'adaptateur fourni dans le kit donne la possibilité d'avoir ces signaux sur un connecteur mâle 9 points "**Standard CIA**".

Le câblage entre le connecteur **J2** et le connecteur 9 points est le suivant:

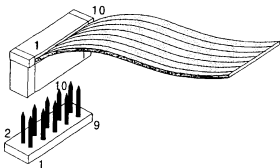
9 points	HE_10
1	1
2	3
3	5
4	7
5	9
6	2
7	4
8	6
9	8
X	10 Non connectée

Cet adaptateur est considéré comme un "stub" s'il n'est pas connecté aux extrémités du réseau. En conséquence, il ne doit pas mesurer plus de 30cm.

Connecteurs "Standard CIA "



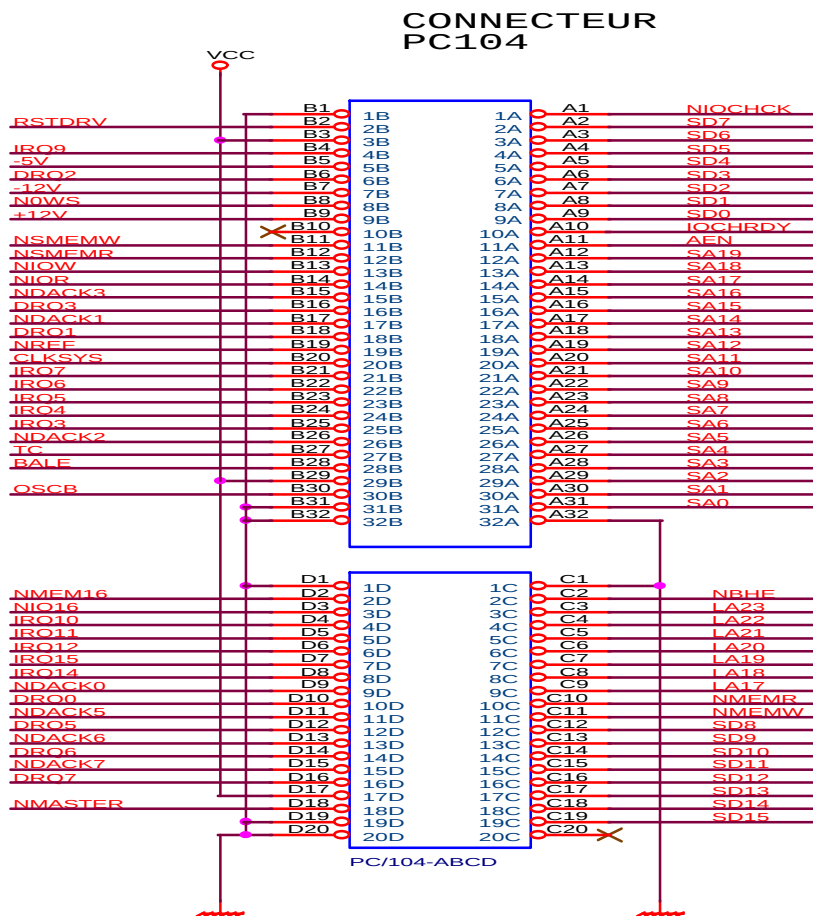
Pin	Signal	Description
1	-	Reserved
2	CAN_L	CAN_L bus line (dominant low)
3	CAN_GND	CAN Ground
4	-	Reserved
5	(CAN_SHLD)	Optional CAN Shield
6	(GND)	Optional Ground
7	CAN_H	CAN_H bus line (dominant high)
8	-	Reserved
9	(CAN_V+)	Optional CAN external positive supply (dedicated for supply of transceiver and optocouplers, if galvanic isolation of the bus node applies)



Pin	Signal	Description
1	reserved	
2	(GND)	Optional Ground
3	CAN_L	CAN_L bus line (dominant low)
4	CAN_H	CAN_H bus line (dominant high)
5	CAN_GND	CAN Ground
6	reserved	
7	reserved	
8	(CAN_V+)	Optional CAN external positive supply
9	reserved	
10	reserved	

Connecteur PC/104

Le brochage du connecteur "J1", PC/104 au format 8 et 16 bits, est conforme aux documents édités par le consortium PC/104.



Connecteur d'extension PC/104

Rangée (A) - côté composant

Rangée (B) - côté soudure

PIN	Nom	Fonction	PIN	Nom	Fonction
1	I/O CHCK	Mem. Parity Error	1	GND	Ground 0V
2	D7	DATA bit D7	2	RESET	System reset
3	D6	DATA bit D6	3	+5V	+5 Volts Power
4	D5	DATA bit D5	4	IRQ9	Interrupt req 9
5	D4	DATA bit D4	5	-5V	-5 Volts Power
6	D3	DATA bit D3	6	DRQ2	DMA Req channel 2
7	D2	DATA bit D2	7	-12V	-12 Volts Power
8	D1	DATA bit D1	8	0WS	Zero Wait State
9	D0	DATA bit D0	9	+12V	+12 Volts Power
10	I/O CHRDY	CPU Ready Control	10	GND	Ground 0V
11	AEN	Address Enable	11	SMEMW	Memory Write (first 1Mb)
12	A19	Address A19	12	SMEMR	Memory Read (first 1Mb)
13	A18	Address A18	13	IOW	I/O Write
14	A17	Address A17	14	IOR	I/O Read
15	A16	Address A16	15	DACK3	DMA Ack channel 3
16	A15	Address A15	16	DRQ3	DMA Req channel 3
17	A14	Address A14	17	DACK1	DMA Ack channel 1
18	A13	Address A13	18	DRQ1	DMA Req channel 1
19	A12	Address A12	19	REF	Memory Refresh
20	A11	Address A11	20	SYSCLK	System clock
21	A10	Address A10	21	IRQ7	Interrupt req 7
22	A9	Address A9	22	IRQ6	Interrupt req 6
23	A8	Address A8	23	IRQ5	Interrupt req 5
24	A7	Address A7	24	IRQ4	Interrupt req 4
25	A6	Address A6	25	IRQ3	Interrupt req 3
26	A5	Address A5	26	DACK2	DMA Ack channel 2
27	A4	Address A4	27	TC	DMA Terminal Count
28	A3	Address A3	28	BALE	Address latch enable
29	A2	Address A2	29	+5V	+5 Volts Power
30	A1	Address A1	30	OSCB	14.31818 MHz clock
31	A0	Address A0	31	GND	Ground 0V
32	GND	Ground 0V	32	GND	Ground 0V

Rangée (C) - côté composant -Bus AT

PIN	Nom	Fonction
1	GND	Ground 0V
2	SBHE	Bus High Enable
3	LA23	Address 23
4	LA22	Address 22
5	LA21	Address 21
6	LA20	Address 20
7	LA19	Address 19
8	LA18	Address 18
9	LA17	Address 17
10	MEMR	Memory Read
11	MEMW	Memory Write
12	SD8	Data bit 8
13	SD9	Data bit 9
14	SD10	Data bit 10
15	SD11	Data bit 11
16	SD12	Data bit 12
17	SD13	Data bit 13
18	SD14	Data bit 14
19	SD15	Data bit 15
20	N.C	No connect

Rangée (D) - côté soudure - Bus AT

PIN	Nom	Fonction
1	GND	Ground 0V
2	MEMCS16	16-bit memory access
3	IOCS16	16-bit I/O access
4	IRQ10	Interrupt Req 10
5	IRQ11	Interrupt Req 11
6	IRQ12	Interrupt Req 12
7	IRQ15	Interrupt Req 15
8	IRQ14	Interrupt Req 14
9	DACK0	DMA Ack channel 0
10	DRQ0	DMA Req channel 0
11	DACK5	DMA Ack channel 5
12	DRQ5	DMA Req channel 5
13	DACK6	DMA Ack channel 6
14	DRQ6	DMA Req channel 6
15	DACK7	DMA Ack channel 7
16	DRQ7	DMA Req channel 7
17	+5V	+5 Volts Power
18	MASTER	Bus Master assert
19	GND	Ground 0V
20	GND	Ground 0V

Carte CAN PC/104 : Connecteurs et cavaliers

LED jaune
Interruption



Cavaliers
d'Adresses

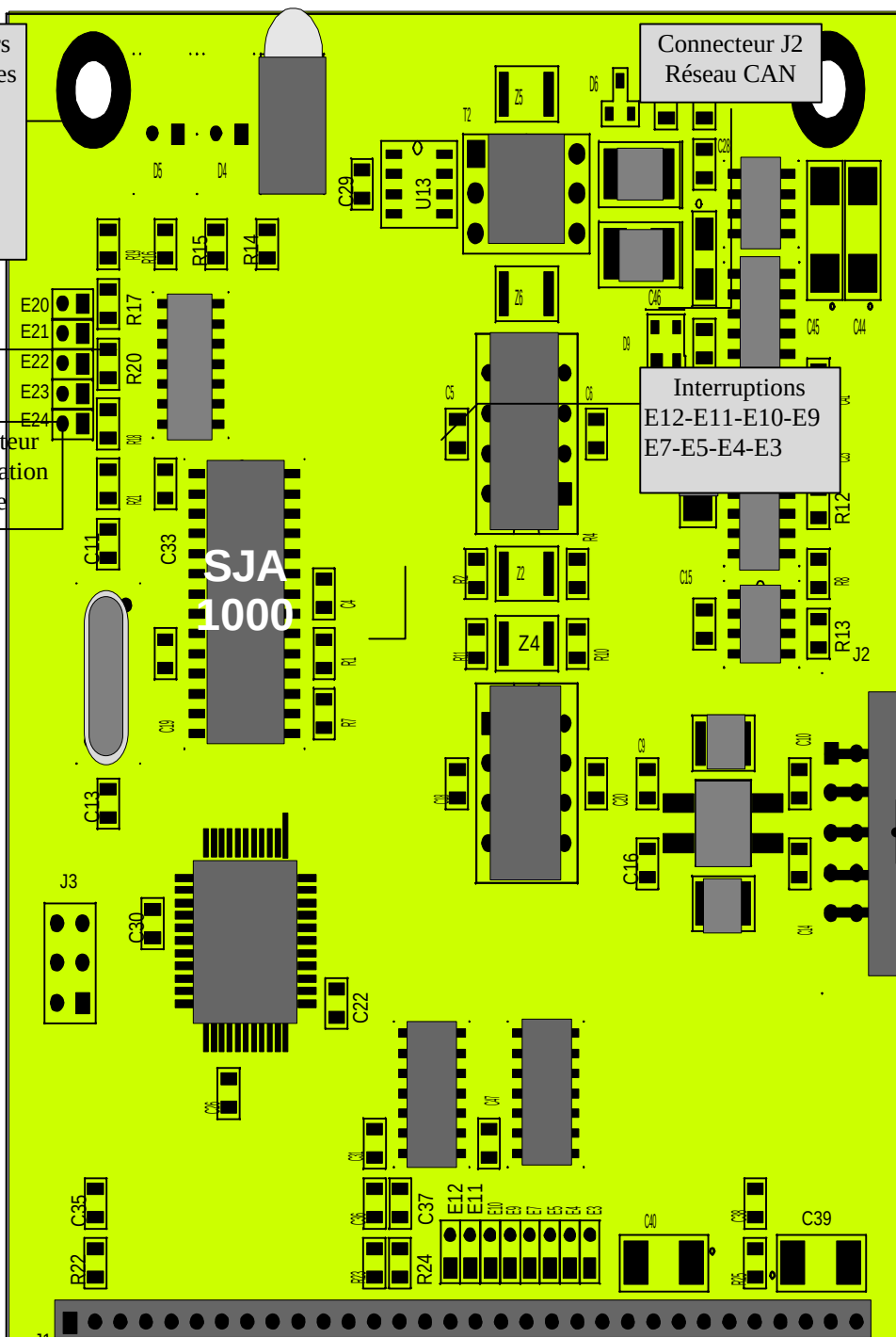
E20
E21
E22
E23
E24

Connecteur
configuration
usine

Connecteur J2
Réseau CAN

Interruptions
E12-E11-E10-E9
E7-E5-E4-E3

SJA
1000



Connecteur J1 PC/1404

INSTALLATION DES DRIVERS ET UTILITAIRES

Les disquettes livrées avec la carte CAN PC/104 contiennent les *drivers* (pilotes de périphériques) facilitant son utilisation dans les environnements MS-DOS et Windows de Microsoft. Des programmes utilitaires permettant de tester la carte CAN PC/104 "en situation" dans ces différents environnements sont aussi fournis.



Un kit de développement permet aux programmeurs C/C++ d'intervenir efficacement sur le bus CAN dans ces mêmes environnements.

MS-DOS

DRIVER

La mise en œuvre de la carte CAN PC/104 sous **MS-DOS** suppose que les drivers **ADTM.EXE** et **ACAN.EXE** soient installés.

ACAN.EXE est le driver de la carte CAN PC/104, conçu sous la forme d'un programme de type TSR (*Terminate and Stay Resident.*) Ce driver gère les accès au bus CAN et offre une interface de programmation aux applications. Pour des raisons d'efficacité, le choix d'un fonctionnement multitâches du driver ACAN a été fait, qui impose la présence préalable d'ADTM.

ADTM.EXE est le moniteur multitâches temps réel d'ATON Systèmes. D'un usage général, il permet à des applications MS-DOS écrites en assembleur ou en C/C++ de fonctionner d'une manière multitâches, entre elles comme avec le système.

ADTM se présente sous la forme de deux fichiers :

- ADTM.EXE code du moniteur multitâches (TSR ou driver)
- ADTM.INI fichier de configuration du moniteur

La description détaillée d'ADTM est disponible dans un autre document.

Tout comme ADTM, ACAN.EXE installe en mémoire le code nécessaire et suffisant à la mise en œuvre de la carte. Il n'a besoin pour fonctionner que de connaître l'adresse de base des entrées/sorties et le numéro de l'interruption utilisée par le composant central de la carte. Sa désinstallation est possible.

INSTALLATION AUTOMATIQUE

L'installation automatique du logiciel se fait en invoquant la commande SETUP.BAT du répertoire DOS de la disquette d'installation selon la syntaxe suivante :

SETUP <port> <irq> [<répertoire>]

SETUP recopie alors les fichiers nécessaires dans le <répertoire> destination (par défaut, **C:\ACAN**), tout en y créant un fichier ADTM.INI minimal et un fichier START.BAT illustrant le lancement du driver ACAN pour la configuration de <port> et d'<irq> choisie.

La commande START peut être utilisée telle quelle ou servir de modèle pour le lancement d'une application plus élaborée.

SETUP crée aussi une commande STOP.BAT, illustrant la séquence de commandes à produire pour désinstaller à la fois ACAN et ADTM.

INSTALLATION MANUELLE

Pour installer le driver ACAN, il faut que le moniteur multitâches ADTM ait été installé au préalable. Dans sa configuration logicielle (fichier ADTM.INI), ADTM doit être préparé à gérer l'interruption matérielle correspondant à la configuration matérielle de la carte CAN PC/104 (ligne "**Irq**s=..."). Le numéro de cette interruption doit aussi apparaître sur la ligne de commande d'ACAN, avec l'adresse de base des entrées/sorties de la carte.

La syntaxe d'invocation d'ACAN.EXE est la suivante :

ACAN /P:<port> /I:<interruption>[*]

installation

ou

ACAN /U

désinstallation

Si le numéro d'interruption est suivi du caractère '*', ADTM sera en mesure de gérer le partage du niveau correspondant avec un autre équipement, si cet autre équipement utilise aussi les services de partage d'interruption d'ADTM.

Exemple d'installation minimale

L'installation du driver pour une carte CAN PC/104 installée à l'adresse d'entrées/sorties **280h** et utilisant la ligne d'interruption 5 se résume au lancement des deux TSR :

C:\>ADTM

C:\>ACAN /P:280 /I:5

en supposant que le fichier ADTM.INI contienne (entre autres) les lignes :

IrqRoutines=1 ; au moins !

Irqs=5 ; IRQ de la carte CAN PC/104

La désinstallation du driver ACAN est possible (option /U) et provoque la libération des ressources ADTM utilisées (l'IRQ) ainsi que celle de la mémoire précédemment allouée.

UTILITAIRES

CANTERM

Le programme CANTERM.EXE est une émulation de terminal rudimentaire mais efficace pour toute machine fonctionnant sous MS-DOS et connectée à un bus CAN par l'intermédiaire de la carte CAN PC/104.

Dans son mode de fonctionnement de base, CANTERM affiche sur l'écran tous les caractères qu'il reçoit du bus CAN et émet sur ce même bus CAN tous les caractères frappés au clavier.

Faute d'être immédiatement utile dans un monde où les terminaux sont encore de préférence connectés à des liaisons série « ordinaires », CANTERM s'avère être un outil de test de bon fonctionnement du bus CAN à la fois simple et efficace : il permet en effet l'envoi et la réception

de messages clairement définis sur le bus CAN, sous contrôle total de l'utilisateur.

Sa syntaxe d'invocation est :

CANTERM [options] <local MID> <remote MID>

Les valeurs de **<local MID>** et de **<remote MID>** sont comprises entre 0 et 7 et doivent être fournies. Elles correspondent respectivement au numéro de la machine où s'exécute localement CANTERM et au numéro de celle où s'exécute son interlocuteur. Dans le cadre de l'application CANTERM, les numéros de machine peuvent être assimilés à des adresses permettant d'identifier formellement les différents interlocuteurs présents sur le bus. Plusieurs machines (au moins 2) sont nécessaires pour mettre en oeuvre une communication valable sur le bus. Si une seule machine est présente, il y a de fortes chances que chaque tentative d'émission de message sur le bus se solde par une erreur, d'ailleurs signalée par CANTERM...

Si le numéro de machine local (**<local MID>**) vaut 0, CANTERM y substituera le numéro de machine courant, pour peu qu'il existe et que son format soit compatible avec celui de l'application CANTERM.

Si le numéro de machine distante (**<remote MID>**) vaut 0, les messages émis par CANTERM seront reçus par toutes les autres machines faisant fonctionner CANTERM.

Les options de CANTERM sont :

/C:<clock>	CAN controller clock (KHz)
/R:<rate>	CAN bus bit rate (Kb/s)
/I	force hardware initialization
/E	echo mode
/L	loop mode

<clock> permet de spécifier la fréquence de l'oscillateur utilisé par le contrôleur CAN de la carte. Sa valeur par défaut est de 16000 KHz, qui correspond à la fréquence du quartz soudé actuellement sur la carte CAN PC/104 : sauf à utiliser CANTERM avec une version ultérieure de la carte, il y a peu de chance que cette option soit utile.

<rate> définit la fréquence d'utilisation du bus CAN pour une application donnée. Cette fréquence doit être identique pour tous les systèmes connectés au bus (par exemple, deux PC équipés de la carte CAN PC/104 et faisant fonctionner CANTERM.) La valeur par défaut de **<rate>** est de 250 Kbits par seconde.

L'option **/I** permet d'imposer l'initialisation du contrôleur CAN pour qu'il puisse fonctionner avec les valeurs de **<clock>** et de **<rate>** en vigueur. Si cette option n'est pas spécifiée, CANTERM n'imposera ses valeurs de **<clock>** et de **<rate>** que dans la mesure où le contrôleur CAN n'est pas déjà initialisé.

L'option **/E** peut être utilisée quand un écho local est désiré. Les caractères frappés au clavier sont alors affichés localement, en plus d'être émis vers la machine distante.

L'option **/L** permet de faire fonctionner CANTERM en mode « bouclage ». Dans ce mode, tous les caractères reçus du CAN sont retournés directement à la machine distante (**<remote MID>**), c'est à dire en général à celle qui l'a émise.

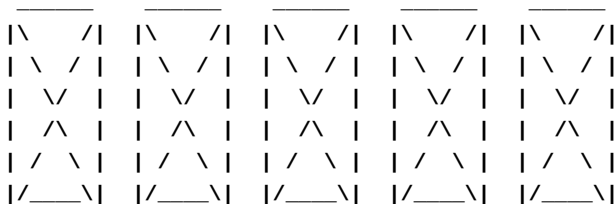
Utilisation

L'utilisation de CANTERM est immédiate : une fois lancé le programme, il suffit de frapper au clavier pour émettre sur le bus CAN. Le programme se termine dès que CTRL-C est frappé.

Dans ce type d'utilisation, et à moins qu'à la fois les capacités de votre clavier et votre vitesse de frappe soient vraiment exceptionnelles, il y a très peu de chance pour que les messages émis contiennent plus d'un caractère...

Cependant, CANTERM se voulant avant tout un programme de test, certaines combinaisons de touches ont une interprétation spéciale :

- La frappe de CTRL-U provoque l'émission des 16 caractères '0123456789ABCDEF'. Selon toute probabilité, cette rafale consistera en l'émission de deux messages CAN de 8 octets chacun.
- La frappe de CTRL-T déclenche quant à elle une rafale plus importante, où les 344 caractères du motif suivant sont émis :



Windows 3.1x

DRIVER

Pour utiliser la carte CAN PC/104 sous **Windows 3.1x**, le VxD (*Virtual Device Driver*) **VACAND.386** doit être installé. Il faut de plus installer la DLL (*Dynamic Link Library*) **ACAN.DLL** pour que les programmes utilitaires et les applications puissent fonctionner.

INSTALLATION AUTOMATIQUE

L'installation automatique du logiciel se fait en invoquant la commande SETUP.EXE du répertoire DRIVERS\WIN3 de la disquette d'installation selon la syntaxe suivante :

SETUP

SETUP se charge alors de poser les questions relatives aux répertoires d'installation des utilitaires et à la configuration matérielle (adresse de base des entrées/sorties et ligne d'interruption de la carte CAN PC/104).

Si l'installation se passe correctement, la carte est utilisable dès le chargement suivant de Windows.

INSTALLATION MANUELLE

Pour installer manuellement le logiciel permettant d'utiliser la carte sous Windows 3.1x, il faut :

1. Recopier les fichiers **VACAND.386** et **ACAN.DLL** du répertoire WIN3 de la disquette d'installation vers le répertoire système de Windows (en général, C:\WINDOWS\SYSTEM.)
2. Ajouter la ligne :

device=vacand.386

dans la section **[386Enh]** du fichier SYSTEM.INI (localisé dans le répertoire Windows, soit en général C:\WINDOWS) et ajouter dans ce même fichier la section entière dont les lignes suivent :

[ACAN]
Base=<port>
Irq=<irq>

<port> est l'adresse de base des entrées/sorties de la carte et **<irq>** est le numéro de la ligne d'interruption utilisée.

Il est essentiel que les valeurs de **<port>** et **<irq>** correspondent à la configuration matérielle imposée par les cavaliers de la carte !

3. Recopier les fichiers des programmes utilitaires **WCT.EXE** et **CANSPY.EXE** du répertoire WIN3 de la disquette d'installation vers le répertoire de son choix (par exemple, C:\ACAN\WIN3.)
4. Si un certain confort d'utilisation est souhaité, créer un groupe de programmes et les programmes correspondant à l'invocation des programmes **WCT.EXE** et **CANSPY.EXE**.
5. Relancer Windows

UTILITAIRES

Windows CAN Terminal (WCT)

Le programme WCT.EXE apporte les mêmes fonctionnalités que son équivalent sous MS-DOS : CANTERM.EXE.

Il permet à toute machine fonctionnant sous Windows et connectée à un bus CAN par l'intermédiaire de la carte CAN PC/104 de se comporter comme un terminal, en affichant sur l'écran tous les caractères qu'il reçoit

du bus CAN et en émettant sur ce même bus tous les caractères frappés au clavier par l'utilisateur.

Pour ses besoins en configuration, WCT utilise par défaut un fichier « .INI » localisé dans le répertoire de travail : WCT.INI. Ce fichier est organisé comme suit :

```
[WCT]
Clock=<clock>
Rate=<rate>
Local=<local MID>
Target=<remote MID>
Echo=0|1
Loop=0|1
```

Les paramètres **Clock** et **Rate** de WCT.INI peuvent être outrepassés en invoquant WCT.EXE de la façon suivante :

WCT /C:<clock> /R:<rate>

La configuration peut être modifiée dans l'application (en passant par le menu **Options/CAN.**)

Le comportement des options de WCT est identique à celles de CANTERM, décrit plus haut, aux variantes suivantes près :

- L'initialisation du contrôleur CAN par WCT est impérative et systématique. Contrairement à ce qui se passe pour CANTERM, WCT impose toujours ses propres valeurs de **<clock>** et de **<rate>** mais aussi de **<local MID>**, en provoquant une réinitialisation complète du contrôleur. Si d'autres applications CAN doivent fonctionner sur la même machine, il est important d'en tenir compte.
- Pour cette raison, le numéro de machine local (**<local MID>**) doit être compris entre 1 et 7, la réactivité aux messages à destination de la machine numéro 0 étant assurée de toutes façons.

Utilisation

L'utilisation de WCT est immédiate : une fois lancé le programme, il suffit de frapper au clavier pour émettre sur le bus CAN. Le programme se termine par la sélection du menu **File/Quit** ou par les autres moyens conventionnels fournis par Windows.

La frappe de CTRL-U et de CTRL-T a les mêmes effets que sous CANTERM.

CAN Spy (CANSPY)

Le programme CANSPY.EXE est un véritable outil de *monitoring* du bus CAN, qui offre aussi la possibilité d'envoyer des messages quelconques sur le bus.

Affichage

CANSPY affiche sous une forme synthétique le contenu des messages qu'il voit passer sur le bus. Une trace typique serait :

CAN Spy - Version 1.0
*** (c) Copyright ATON Systèmes 1998

ACAN.DLL Version 1.0
VACAND Driver Version 1.0
Philips SJA1000 CAN Controller detected
Hardware was previously initialized
Clock: 16000 KHz, Rate: 250 Kbps
TSEG1 = 11, TSEG2 = 5, SWJ = 3, MSI = 1

```
10:04:01 - Hardware is initialized
10:04:01 - TRACE IS ON
10:04:17 - 00001001000 -          61          | a
10:04:17 - 00001001000 -          62          | b
10:04:17 - 00001001000 -          63          | c
10:04:18 - 00001001000 -      30 31 32 33 34 35 36 37 | 01234567
10:04:18 - 00001001000 -      38 39 41 42 43 44 45 46 | 89ABCDEF
10:04:37 - 00001001000 -          68          | h
10:04:37 - 00001001000 -          65          | e
10:04:37 - 00001001000 -          6C          | l
10:04:37 - 00001001000 -          6C          | l
10:04:37 - 00001001000 -          6F          | o
```

Chaque message reçu est daté. Il est à noter que cette datation correspond au moment où CANSPY affiche le message, qui peut être quelque peu

éloigné de celui où le message a été émis. Quoi qu'il en soit, la chronologie de la réception (et par conséquent celle de l'émission) est respectée.

Les identificateurs des messages sont affichés en binaire, sur 11 bits pour les messages standard et sur 29 bits pour les messages étendus. Cette notation a été préférée à une notification hexadécimale pour la meilleure qualité des informations qu'elle apporte dans le cadre d'une application CAN, où la position et la teneur des bits d'identificateurs sont généralement très importantes.

Les messages dont le bit RTR (*Remote Transmission Request*) est à 1 peuvent être différenciés des autres par l'affichage d'un 'R' après l'identificateur.

S'il y a lieu, chaque octet de donnée est affiché sous la forme de deux chiffres hexadécimaux et sous celle d'un caractère ASCII.

Contrôle

CANSPY garde en mémoire les 800 derniers messages reçus. Si les informations de trace doivent être conservées, il est possible de les sauvegarder dans un fichier texte à l'aide de **File/Redirect to...**

La trace peut être interrompue et reprise par **Control/Trace**.

Elle peut être nettoyée par **Control/Clear**.

Envoi de messages

Il est possible d'émettre ponctuellement un message à l'aide de **Control/Send**.

La teneur du message doit être spécifiée en utilisant la même notation qu'à l'affichage.

CANSPY conserve une liste des 10 derniers messages émis, ce qui peut faciliter le travail de mise au point...

Remarque importante

Si d'autres applications CAN doivent coexister avec CANSPY sur la même machine, il est extrêmement important que CANSPY soit lancé en dernier. CANSPY modifie en effet le filtre à messages du contrôleur CAN de façon à être en mesure de récupérer tous les messages qui transitent sur le bus.

Quand une application initialise le contrôleur pour définir la vitesse de transfert, le type de drivers de bus utilisés, etc., elle met aussi à jour le filtre

hardware correspondant au numéro de machine local. Dans le cadre d'une application normale, ce filtre matériel effectue un travail extrêmement efficace en évitant au driver de travailler pour des messages qui ne concernent pas la « machine » locale, au sens ACAN du terme.

Quand CANSPY s'installe, les effets bénéfiques de ce premier filtrage sont annihilés car il impose le filtrage matériel « zéro » qui lui permet de voir passer la totalité des messages qui transitent sur le bus. Cela peut donc avoir un impact sur la performances générale des autres applications déjà actives sur la même machine, principalement si le trafic sur le bus est important.

Les applications ACAN qui doivent s'installer après d'autres applications ACAN ne peuvent s'accommoder des initialisations matérielles précédentes que si la structure générale du filtrage du numéro de machine reste constante. Une fois CANSPY installé cependant, bien peu d'application seront capables de supporter le format imposé par cet utilitaire, trop tolérant à bien des titres...

Windows 95

DRIVER

Pour utiliser la carte CAN PC/104 de la façon la plus efficace sous **Windows 95**, le VxD (*Virtual Device Driver*) **VACAND.VXD** doit être installé. Il faut de plus installer la DLL (*Dynamic Link Library*) **ACAN32.DLL** pour que les programmes utilitaires et les applications puissent fonctionner. L'utilisation de VACAND.VXD et d'ACAN32.DLL permet aux applications 32 bits (dont les utilitaires WCT32 et CANSPY32) de fonctionner en tirant partie de la puissance de l'interface de programmation Win32. Afin de pouvoir continuer à utiliser les applications 16 bits, l'interface de VACAND.VXD avec ACAN.DLL a été préservée.

Noter qu'il est possible d'utiliser le VxD **VACAND.386** et la DLL 16 bit ACAN.DLL sous Windows 95. Il faudra alors se contenter des versions 16 bits des utilitaires Windows CAN Terminal et CAN Spy (WCT.EXE et CANSPY.EXE.)

INSTALLATION AUTOMATIQUE

L'installation automatique du logiciel se fait en invoquant la commande SETUP.EXE de la disquette d'installation spécifique à Windows 95 selon la syntaxe suivante :

SETUP

SETUP se charge alors de poser les questions relatives à la configuration souhaitée (adresse de base des entrées/sorties et ligne d'interruption de la carte CAN PC/104) et aux répertoires d'installation des utilitaires.

Si l'installation se passe correctement, la carte est utilisable dès le chargement suivant de Windows.

INSTALLATION MANUELLE

Toute tentative d'installation manuelle du driver est fortement déconseillée et trop complexe pour être décrite dans le cadre de ce manuel.

UTILITAIRES

Les utilitaires WCT32.EXE et CANSPY32.EXE sont les versions 32 bits des utilitaires 16 bits WCT.EXE et CANSPY.EXE, décrits précédemment. Leurs fonctionnalités sont identiques.

PROGRAMMATION

Ce chapitre décrit les interfaces permettant aux programmes d'utiliser la carte CAN PC/104 par l'intermédiaire des pilotes **ACAN**. L'emphasis a été mise sur la programmation en C/C++. L'interface de programmation en assembleur, qui ne concerne que MS-DOS, est fournie en annexe.



GENERALITES

Le CAN par ATON

L'utilisation de la carte CAN PC/104 par des applications se conformant aux différents standards en matière de communication CAN¹ est tout à fait possible, bien qu'ATON Systèmes ne fournisse pas les couches logicielles implémentant ces standards.

Dans le cadre des applications CAN verticales, où les nœuds intervenant sur le bus sont bien connus et où les messages qu'ils échangent sont bien définis, ATON Systèmes propose sa propre façon de faire, à la fois simple et efficace. Pour cette utilisation du CAN, un certain nombre de règles définissent la façon dont les messages doivent être identifiés sur l'ensemble d'un système CAN opérationnel (bus et intervenants).

Identification des messages

Le principe général est que les différents acteurs présents sur le bus CAN possèdent un numéro unique permettant de les identifier². Tous les messages transitant sur le bus sont destinés soit à l'un de ces acteurs, soit à l'ensemble des acteurs présents (*broadcast*.) Pour ce faire, quelques bits de l'identificateur CAN des messages sont réservés au "numéro de cible",

¹ Comme par exemple CANOpen, Device Net, ou SDS.

² Ce principe n'est pas vraiment révolutionnaire dans la mesure où on le retrouve dans les standards mentionnés plus haut, avec sans doute plus de sophistication dans le cas de Device Net.

aussi appelé MID pour *Machine ID*. Idéalement, ces bits devraient être positionnés de façon à ce que l'utilisation indifférente de messages "standard" (11 bits, CAN 2.0a) ou "étendus" (29 bits, CAN 2.0b) soit possible.

Les identificateurs et les masques d'identificateurs utilisés par les logiciels qui conversent avec les drivers ACAN ont tous l'apparence de mots de 32 bits :

```

          3 3 2 2 2 2 2 2 | 2 2 2 2 1 1 1 1 | 1 1 1 1 1 1
Bit :      1 0 9 8 7 6 5 4 | 3 2 1 0 9 8 7 6 | 5 4 3 2 1 0 9 8 | 7 6 5 4 3 2 1 0

          2 2 2 2 2 | 2 2 2 2 1 1 1 1 | 1 1 1 1 1 1
ID étendu : 8 7 6 5 4 | 3 2 1 0 9 8 7 6 | 5 4 3 2 1 0 9 8 | 7 6 5 4 3 2 1 0

          1
ID standard : 0 9 8 7 6 | 5 4 3 2 1 0

ID ACAN:  0 0 0 a b c d e | f g h i j k l m | n o p q r s t u | v w x y z α β γ

```

Dans cet exemple, le MID est codé sur les bits ID2, ID1 et ID0 des identificateurs standard, ce qui correspond aux bits ID20, ID19 et ID18 des identificateurs étendus. Noter que sur le bus CAN, les bits de l'identificateur arrivent dans l'ordre a, b, c..., c'est-à-dire ID10, ID9, ID8... pour les identificateurs standard ou ID28, ID27, ID26... pour les identificateurs étendus. Les 3 bits codant ici pour le MID permettent de définir des systèmes comportant jusqu'à 7 intervenants numérotés de 1 à 7, l'adresse 0 étant par définition réservée aux messages *broadcast*.

En pratique, le nombre de bits utilisés pour le MID n'est pas figé, tout comme n'est pas figée la position de ces bits dans l'identificateur des messages : la seule contrainte apportée par ce type d'organisation des identificateurs de messages est que l'adresse de la destination est spécifiée dans le message.

Tout comme pour le MID, la définition de l'adresse *broadcast* (BID pour *Broadcast ID*) n'est pas figée en ce qui concerne le nombre et la position des bits dans un message. Par contre, il est impératif que tous les bits participants à cette adresse soient à 0.

Bien que cela ne soit pas forcément conseillé, il est possible de définir des organisations différentes pour le MID et le BID au sein d'un même système.

Filtrage des messages

Les messages sont filtrés au niveau de leur identificateur.

Les messages reçus peuvent être répartis en 2 groupes : les messages ciblés, destinés à la machine hôte et les messages *broadcast*, destinés à toutes les machines.

Autant que faire se peut, le matériel assure au moins le premier niveau de filtrage en ne prenant en compte que les messages dont l'identificateur porte le MID de l'hôte ou le BID. Le contrôleur Philips SJA1000 de la carte CAN PC/104 possède deux *acceptance masks* qui permettent ce filtrage et réduisent considérablement le nombre de fausses alertes qui se produiraient si tel n'était pas le cas.

Le logiciel du driver assure la finition.

Comme pour la plupart des contrôleurs CAN un peu évolués, le filtrage des messages par logiciel fait appel à deux masques appliqués bit à bit à l'identificateur de chaque message reçu, c'est-à-dire de chaque message ayant été sélectionné par le filtre matériel. La première valeur appliquée est un masque AND (ET logique), permettant de ne retenir que les bits significatifs de l'identificateur testé. La seconde valeur est un masque XOR (OU Exclusif), appliqué au résultat du AND précédent. Le message est retenu si le résultat est 0 (ce qui est équivalent à une comparaison après masquage).

Pour éviter l'accumulation de messages inutiles, la routine de traitement de l'interruption du contrôleur CAN assure le filtrage terminal dès la réception d'un message. Les messages pour lesquels les applications ont signalé un intérêt auprès du driver sont donc recopiés dans les tampons de ces applications dès leur réception³. Noter qu'un message peut être ainsi reçu par plusieurs applications.

Canaux CAN

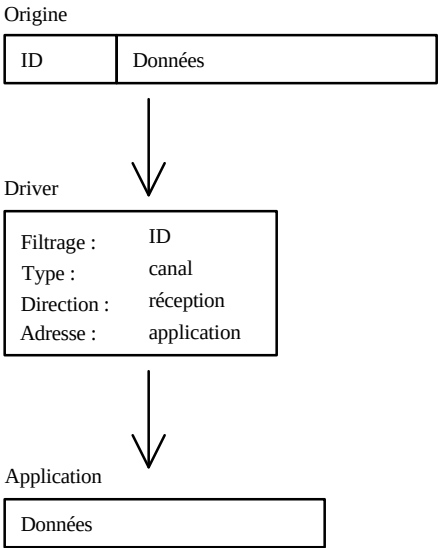
La notion de "canal CAN" est une autre particularité d'ATON, dont l'utilisation est optionnelle et sans impact sur le fonctionnement général du système. Les canaux CAN peuvent s'avérer extrêmement efficaces quand les applications impliquées dans un système CAN doivent effectuer la mise à jour ou la consultation répétitives de variables d'état ou de contrôle sur des cibles distantes.

³ Sous Windows, cette recopie peut être très légèrement différée.

Un canal CAN permet à une application située sur une machine de modifier ou de consulter à moindre coût la valeur d'une variable détenue par une autre application, sur une autre machine. Pour être efficace, la consultation ou la mise à jour se fait avec le minimum de coopération de l'application qui détient la variable canal. Par définition, la taille d'une variable canal ne peut pas excéder 8 octets.

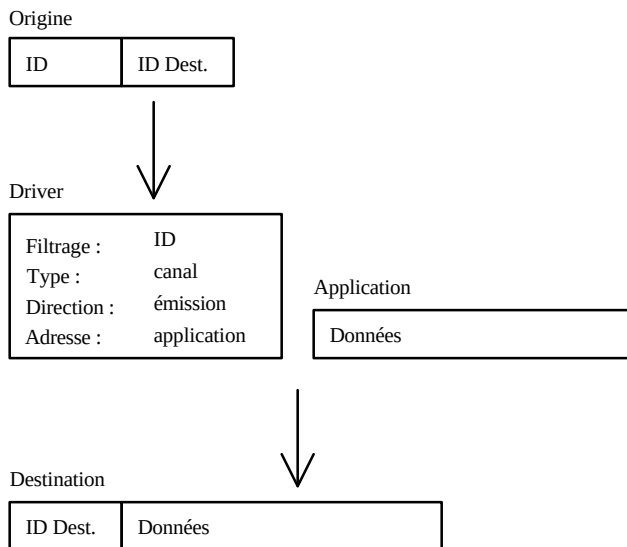
Quand elle fait enregistrer un canal par le driver (voir plus loin), l'application hôte spécifie un filtre à identificateur (généralement unique), une adresse mémoire, une taille et un sens de transfert. Quand un message est reçu pour l'identificateur spécifié, le driver se comporte différemment selon le sens du transfert spécifié à l'enregistrement.

En réception, les données du message sont recopiées par le driver à l'adresse indiquée lors de l'enregistrement, sur une longueur correspondant au minimum du nombre d'octets de données effectivement reçus et de la longueur enregistrée. Aucune intervention de l'application n'est nécessaire.



En émission, le message reçu initialement porte avec lui l'identificateur, le format et la longueur des données du message utilisé pour émettre les données demandées. Le driver émet automatiquement ce message, garni avec les données de l'application, sans qu'aucune intervention de l'application ne

soit nécessaire. Il peut être intéressant de noter que les données ne sont pas forcément retournées à la machine ayant émis la demande de transmission. On peut aussi noter que l'identificateur de la destination peut correspondre à celui d'un autre canal CAN orienté en réception.



Le principal intérêt des canaux est la performance, le bus CAN et le driver travaillant à l'arrière-plan sans que les applications ne soient perturbées en aucune mesure. Un autre avantage non négligeable réside dans la simplicité de leur mise en œuvre, qui ne nécessite a priori aucune programmation multitâches...

Fonctionnement général des drivers et des applications

La mise en œuvre du CAN via ACAN se fait de la façon suivante :

- Le driver récupère les coordonnées physiques du contrôleur CAN (adresse de base, numéro d'interruption) par les moyens les plus appropriés (paramètres de la ligne de commande sous MS-DOS, SYSTEM.INI sous Windows 3.1x, configuration Plug and Play sous Windows 95, etc.).
- Le driver s'assure de la présence du contrôleur CAN et refuse de s'installer s'il n'en trouve pas.

- Le driver met le contrôleur CAN dans un état où il est insensible à ce qui se passe sur le bus (*Bus Off*).
- Le driver prend le contrôle de l'interruption du contrôleur CAN.
- Le driver se met en disponibilité pour accepter les commandes des applications.
- L'application utilisatrice s'assure de la présence du driver.
- L'application interroge le driver sur son état. Si le matériel n'a pas encore été initialisé ou si ces initialisations ne lui conviennent pas, elle fournit au driver tous les paramètres nécessaires à cette fin (fréquence d'horloge, taux de transfert souhaité, caractéristiques de l'échantillonnage des bits CAN, type d'interface avec le bus, etc.) Les masques permettant de définir les numéros de machine (MID et BID) sont aussi passés au driver, qui peut mettre à jour les filtres matériels nécessaires à la bonne réception des messages qui concernent cette machine.
- L'application enregistre auprès du driver les "conditions spéciales" (erreurs, événements exceptionnels, etc.) pour lesquelles elle souhaite être notifiée ainsi que les identificateurs des messages pour lesquels elle souhaite un traitement particulier. Ces traitements peuvent consister en une recopie automatique de données entre la mémoire de l'utilisateur et le message (canaux CAN) ou en une notification d'arrivée de messages à l'attention de l'application.
- L'application demande au driver de passer *Bus On*. Le driver devient alors sensible à ce qui peut se passer sur le bus et devient capable de notifier aux applications intéressées les événements significatifs, comme l'occurrence d'une condition spéciale ou la réception d'un message. Le mécanisme exact de la notification des événements peut revêtir des apparences diverses selon le système d'exploitation utilisé.
- L'occurrence d'une condition spéciale provoque la notification par le driver des applications qui se sont enregistrées à cet effet.
- La réception de messages par l'application se fait toujours en deux temps : notification de réception par le driver puis récupération des messages en attente par l'application.
- L'émission d'un message par l'application est plus immédiate : le message est passé directement au driver, qui l'émet dès que l'état du bus le permet, après une mise en liste d'attente éventuelle. Les messages tamponnés sont rangés (donc émis) en fonction d'un niveau de priorité spécifié par l'application. Les messages émis avec la priorité

0 (la plus forte priorité) sont rebouclés en interne par le driver, en plus d'être émis sur le bus CAN.

Ce mode de fonctionnement se veut général et donne en pratique toute satisfaction sur les systèmes où il a été implémenté à ce jour.

La généralisation a été poussée à tel point que l'interface de programmation avec les bibliothèques chargées de communiquer avec les drivers "ACAN" est quasiment identique et qu'un fichier de définitions unique (**ACAN.H**) est utilisé pour tous les systèmes.

STRUCTURES ESSENTIELLES.

L'interface de programmation fait appel à quelques structures essentielles qui, à quelques variantes près, sont identiques en composition et en taille pour tous les systèmes.

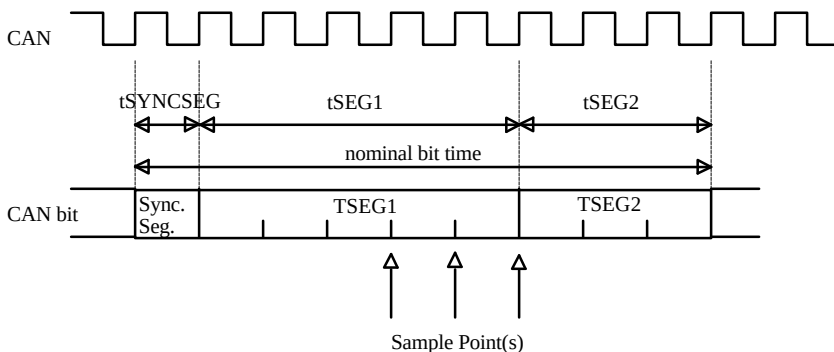
Définition de la configuration matérielle

Cette structure est utilisée pour mettre à jour ou pour récupérer la configuration courante du contrôleur CAN (voir **ACAN_SetHardConfig** et **ACAN_GetHardConfig**, plus loin dans ce chapitre).

```
typedef struct tACANH1 {
    DWORD    hi_miv;    /* machine ID value */
    DWORD    hi_mim;    /* machine ID mask (1 = significant) */
    DWORD    hi_biv;    /* broadcast ID value (should be all 0's) */
    DWORD    hi_bim;    /* broadcast ID mask (1 = significant) */
    WORD     hi_clk;    /* basic controller clock rate (KHz) */
    WORD     hi_rat;    /* CAN bus speed (Kbits/s) */
    BYTE     hi_ts1;    /* TSEG1 (bit clock quanta) */
    BYTE     hi_ts2;    /* TSEG2 (bit clock quanta) */
    BYTE     hi_sjw;    /* sync. jump width (bit clock quanta) */
    BYTE     hi_msi;    /* multiple samples indicator */
    BYTE     hi_hsb[8]; /* more, depending upon the controller type */
} ACANH1, *PACANH1;

/* For Stand-alone CAN Controller (CANC_SJA1000): */
#define hi_ocr    hi_hsb[0] /* Output Control Reg. value */
#define hi_ien    hi_hsb[1] /* Interrupt Enable Reg. value (if != 0) */
```

La norme CAN définit le bit de donnée élémentaire qui transite sur le bus comme la somme de *quanta* de temps identiques. Cette décomposition arbitraire permet de définir les "instants significatifs" de la vie du bit :



Le point le plus utile concrètement est le "point d'échantillonnage" du bit CAN (*sample point*), qui correspond à l'instant où le contrôleur échantillonne sa valeur récessive ou dominante. La norme (et le bon sens !) préconise un échantillonnage situé environ aux 2/3 de la durée totale du bit, c'est-à-dire à un moment où à la fois le bit est stable et où il y a peu de chances qu'il soit terminé.

La possibilité d'échantillonnages multiples (*multiple sample*) renforce la fiabilité de l'ensemble. L'échantillonnage multiple est recommandé sur les bus à basse vitesse (classes A et B) fonctionnant dans un environnement parasite. Il est déconseillé pour les bus à haute vitesse (classe C).

Pour s'accommoder de la disparité probable des horloges des différents intervenants, la plupart des contrôleurs CAN évolués autorisent un certain décalage entre le point de départ théorique du bit et la réalité. La "largeur du saut de synchronisation" (*Synchronization Jump Width*) précise le nombre de quanta maximum pendant lesquels le contrôleur ne s'offusquera pas de voir changer la valeur du bit après son début théorique. Les contrôleurs profitent d'ailleurs des changements de valeurs des bits pour se resynchroniser sur la véritable activité du bus.

La vitesse de fonctionnement du bus CAN détermine la durée du bit. Il est cependant possible de faire varier par programme le nombre de quanta que contient le bit, de fixer le ou les points d'échantillonnage ainsi que la largeur du saut de synchronisation (*Synchronization Jump Width*).

Pour programmer le contrôleur CAN, on suppose qu'il est cadencé par une horloge externe fixe. La durée du quantum de bit CAN découle de la division de cette horloge de base par une valeur programmable, déduite des paramètres passés avec la structure **ACANH1**.

Le contrôleur Philips SJA1000 permet de programmer le fonctionnement de ses entrées/sorties CAN (notées RX0, RX1 et TX0, TX1.) Sur la carte CAN PC/104, seule l'entrée RX0 est utilisée et les drivers programment le composant dans ce sens. La programmation de l'*Output Control Register* (OCR) peut varier selon de type de *driver de ligne*⁴ utilisé. Avec son équipement standard, il est recommandé de programmer OCR avec la valeur 0x6A, passée dans le champ *hi_ocr* (alias pour *hi_hsb[0]*) de la structure **ACANH1**. Cela correspond à un fonctionnement de la paire TX0, TX1 en mode normal (les bits ne changent pas de niveau pendant toute leur durée), vite relayé par un *driver Pull-Down* de type 82C251; avec ce driver, le bit dominant est à 0 volts et le bit récessif à 5 volts.

Toujours avec ce contrôleur, il est possible de passer dans le champ *hi_ien* (alias pour *hi_hsb[1]*) de la structure **ACANH1** les bits correspondant à la programmation du registre d'autorisation des interruptions (*Interrupt Enable Register*), organisé comme suit :

Bit	Symbole	Interruption	Interruption quand...
7	BEIE	Bus Error	Erreur de bus détectée.
6	ALIE	Arbitration Lost	Echec à l'arbitrage (émission en échec).
5	EPiE	Error Passive	Transition entre les états <i>Error Passive</i> et <i>Error Active</i> ⁵ .
4	WUiE	Wake Up	Réveil du contrôleur ⁶ .
3	DOiE	Data Overrun	Perte de messages en réception, par manque de tampons internes.
2	EiE	Error Warning	Au moins l'un des compteurs d'erreur du contrôleur atteint ou dépasse la limite standard de 96, éventuellement

⁴ Il s'agit là d'un composant électronique, à ne pas confondre avec un *driver logiciel* !

⁵ Reportez-vous à la norme pour plus d'informations.

⁶ La gestion de l'énergie n'est pas implémentée actuellement par les drivers.

			jusqu'au point où le contrôleur passe <i>Bus Off</i> .
1	TIE	Transmit	Le contrôleur est prêt à émettre.
0	RIE	Receive	Le contrôleur a reçu au moins un message.

Tous ces bits sont actifs à 1.

Si *hi_ien* est à 0, IER est programmé avec la valeur par défaut 10101101b, qui correspond à l'autorisation des interruptions Bus Error, Error Passive, Data Overrun, Error Warning et Receive.

Les champs *hi_miv*, *hi_mim*, *hi_biv* et *hi_bim* de la structure **ACANH**I sont utilisés pour mettre à jour les numéros de machine (MID) et *broadcast* (BID), ainsi que les filtres matériels correspondant. Les filtres du SJA1000 sont organisés de telle façon que seuls les 16 bits les plus significatifs des valeurs passées soient opérationnels (les bits **abcdefghijklmnop** en notation ACAN.) Dans le cas de la réception d'identificateurs étendus (29 bits), cela correspond au filtrage matériel des bits ID28 à ID13. Pour les identificateurs standard, le filtrage opère sur la totalité des 11 bits d'identificateur (**abcdefghijk** pour ID10 à ID0) et sur le bit *Remote Transmission Request* (1 pour RTR.) S'il y a lieu ce filtrage des messages standard continue sur les 8 bits du premier octet de données, selon des modalités assez particulières (bits **mnp** de *hi_miv* et *hi_mim* pour le quartet de forts poids, bits **mnp** de *hi_biv* et *hi_bim* pour le quartet de faibles poids) !

Les bits à 1 de *hi_mim* et *hi_bim* déterminent les bits de messages à comparer avec les bits équivalents de *hi_miv* et *hi_biv* pour satisfaire le premier niveau de filtrage, matériel.

Bien qu'il soit possible de donner des structures différentes à *hi_mim* et à *hi_bim*, cela devrait être interdit dans le cadre d'applications respectant la philosophie d'adressage ACAN. Dans ce même cadre, *hi_biv* (filtre des messages *broadcast*) devrait toujours valoir 0.

Les drivers exploitent la structure de la façon suivante pour garnir les registres du contrôleur Philips SJA1000 qui équipe la carte CAN PC/104 (reportez-vous à la documentation de ce composant pour plus d'explications) :

⁷ Reportez-vous à la norme pour plus d'informations

Registre	Programmation	Bornes	Commentaire
TSEG1	hi_ts1 - 2	0..15	hi_ts1 = tSYNC + tTSEG1
TSEG2	hi_ts2 - 1	0..7	hi_ts2 = tSEG2
SJW	hi_sjw - 1	0..3	force dans ces bornes
SAM	hi_msi > 1	0..1	
BRP	hi_clk / (hi_rat * 2 * (hi_ts1 + hi_ts2)) - 1	0..63	hi_ts1 + hi_ts2 = quanta par bit CAN
ACR0	hi_miv >> 13	-	abcdefgh
ACR1	hi_miv >> 5	-	ijklmnop
ACR2	0		
ACR3	0		
AMR0	hi_mim >> 13	-	abcdefgh
AMR1	hi_mim >> 5	-	ijklmnop
AMR2	hi_bim >> 13	-	abcdefgh
AMR3	hi_bim >> 5	-	ijklmnop
OCR	hi_hsb[0]	(0x6A)	Output Control Register
IER	hi_hsb[1]		Interrupt Enable Register Si différent de 0

Les vérifications suivantes sont effectuées :

hi_ts1 ∈ [2..17]	TSEG1 ∈ [0..15]
hi_ts2 ∈ [1..8]	TSEG2 ∈ [0..7]
hi_rat != 0	évite les divisions par 0
hi_clk >= hi_rat * 2 * (hi_ts1 + hi_ts2)	BRP >= 0
BRP <= 63	
SJW est forcé dans les bornes [0..3]	

Messages ACAN

Cette structure définit l'organisation des messages CAN échangés entre les applications et les drivers ACAN.

```
typedef struct tACANMB {
    DWORD    dwId;                /* message ID */
    BYTE     bFormat;             /* Frame Format (EF RTR 0 0 0 0 0 0) */
    BYTE     bLength;             /* message data length (0..8) */
}
```



```

    BYTE    bData[8];          /* message data buffer */
} ACANMB, *PACANMB;
#define ACANMSZ sizeof(ACANMB)
/* bFormat bits */
#define FF_EXTENDED    0x80    /* Extended Frame */
#define FF_RTR         0x40    /* Remote Transfer Request */

```

Les noms utilisés et les commentaires en rapport sont suffisamment explicites pour ne pas appeler d'autres commentaires, du moins pour les anglophones ;-)

Noter cependant que l'identificateur du message, *dwID*, respecte la notation introduite précédemment :

ID ACAN: 0 0 0 a b c d e | f g h i j k l m | n o p q r s t u | v w x y z α β χ

Les bits non significatifs des identificateurs des messages standard (bits **lmnopqrstuvwxyzaβχ**) sont passés à 0 par les drivers et devraient aussi être passés à 0 par les applications.

Filtres à messages

Cette structure définit l'organisation des filtres à messages CAN passés aux drivers par les applications à la suite de la structure **ACANTB**, décrite plus loin.

```

typedef struct tACANID {
    DWORD    dwAnd;             /* filtering AND mask */
    DWORD    dwXor;             /* filtering XOR mask */
    BYTE     bAction;           /* actions to be taken by the driver */
    BYTE     bLength;           /* channel transfer length */
    BYTE     bReserved[2];      /* reserved */
    LPBYTE   lpAddress;         /* channel transfer address */
} ACANID, *PACANID;
#define ACANIDSZ sizeof(ACANID)
/* bAction values: "what is the driver supposed to do when it receives
 * a message that can be filtered by dwAnd and dwXor?" */
#define IDA_NOTIFY      0x80    /* notification */
#define IDA_QUEUEMSG    0x40    /* buffering */
#define IDA_CHANRCV     0x20    /* channel data reception */
#define IDA_CHANSND     0x10    /* channel data emission */

```

Comme pour les filtres matériels, les bits à 1 de *dwAnd* déterminent les bits de messages à comparer avec les bits équivalents de *dwXor* pour satisfaire ce deuxième niveau de filtrage.

Voici quelques exemples de filtres utilisables :

Filtrage	<i>dwAnd</i>	<i>dwXor</i>
Tous les messages	0x00000000	0x00000000
Un seul message, d'identificateur <ID>	0x1FFFFFFF	<ID>
Les messages du type 0x01C0XXXX	0x1FFF0000	0x01C00000

Quand un message est retenu par le filtre, *bAction* indique au driver quelles actions il doit mener.

La notification de réception (IDA_NOTIFY) est sans doute l'action la plus importante, qui sera plus amplement décrite par la suite.

Si le bit IDA_QUEUEMSG est à 1, chaque message retenu par le filtre sera tamponné dans la structure ACANTB de l'application, dans la mesure où il y a la place.

Si l'un des deux bits IDA_CHANRCV ou IDA_CHANSND est à 1, le filtre définit un canal ACAN, orienté en réception (IDA_CHANRCV) ou en émission (IDA_CHANSND.) Le champ *lpAddress* est alors significatif et indique l'adresse mémoire où doivent être recopiées les données reçues du CAN pour ce canal (IDA_CHANRCV), ou bien celle où se trouvent les données à émettre sur le CAN (IDA_CHANSND).

Noter que toutes les combinaisons de *bAction* dans lesquelles les bits IDA_CHANRCV et IDA_CHANSND ne sont pas tous les deux à 1 sont permises.

Tampon de communication (*Task Block*)

Cette structure est essentielle pour la communication entre les applications et les drivers ACAN, et tout particulièrement en ce qui concerne les mécanismes de notification des événements CAN aux applications.

Une application qui souhaite recevoir des messages particuliers et être prévenue de l'occurrence de conditions spéciales sur le bus CAN doit enregistrer au moins une structure **ACANTB** auprès du driver, à l'aide de la fonction **ACAN_Register** décrite plus loin dans ce document. Dans cette structure, l'application indique à quels événements elle va être sensible,

quels messages l'intéressent et où les ranger, de quelle façon être notifiée par le driver, etc.

```
typedef struct tACANTB {
    DWORD        dwReserved0;    /*D next ACANTB in driver list */
    /* Notification */
    #if defined(_DOS)
        WORD        wTid;        /*I ADTM TID */
        WORD        wEvent;      /*I ADTM event code */
    #elif defined(WIN16)
        HWND        hWnd;        /*I Win16 window handle */
        UINT        nMsg;        /*I Win16 message ID */
    #else /* assume WIN32 */
        HANDLE       hEvent;      /*I Win32 event handle */
    #endif
    ACANCBFN      cbNotification; /*I callback routine */
    DWORD        dwRefData;      /*I associated data */
    BYTE         bNotType;       /*I notification type */
    BYTE         bSCFilter;      /*I special conditions filter */
    WORD         wStatus;        /*R pending events and Special Cond. */
    BYTE         bREC;           /*R Receive Error Counter (CANC) */
    BYTE         bTEC;           /*R Transmit Error Counter (CANC) */
    BYTE         bALC;           /*R Arbitration Lost Capture (SJA1000) */
    BYTE         bECC;           /*R Error Code Capture (SJA1000) */
    /* Messages */
    LPVOID        lpRMB;         /*I received messages FIFO address */
    WORD          wRMBSize;      /*I FIFO size, in messages */
    WORD          wRMBOccupation; /*R currently buffered messages */
    WORD          wRIndex;       /*D/R FIFO read index */
    WORD          wWIndex;       /*D/R FIFO write index */
    LPVOID        lpBuf;        /*I user message buffer */
    WORD          wCount;        /*I user message buffer size */
    /* Filters and miscellaneous */
    BYTE          bNID;          /*I # ACANID that follow */
    BYTE          bReserved0;     /*D reserved for driver use */
    DWORD         dwReserved1;    /*D reserved for driver use */
    DWORD         dwReserved2;    /*D used by the driver */
    DWORD         dwReserved3;    /*D used by the driver */
    DWORD         dwFree;         /*A free for application use */
    DWORD         dwReserved4;    /*L used by the library */
    /* ACANID      idTable[0]; */ /*I table of ID filters */
} ACANTB, *PACANTB;
```

```
#define ACANTBSZ sizeof(ACANTB) /* without following ACANID */
```

Les champs marqués 'D' sont réservés aux drivers.

Les champs marqués 'L' sont réservés aux bibliothèques d'interfaçage avec les drivers.

Les champs marqués 'A' sont utilisables par les applications.

Les champs marqués 'T' doivent être initialisés par les applications avant l'enregistrement de la structure auprès du driver (voir plus loin).

Les champs marqués 'R' sont mis à jour par les drivers, à l'intention des applications.

A l'enregistrement, la structure est immédiatement suivie des **ACANID** décrivant les filtres des messages qui intéressent l'application. Le champ *bNId* donne le nombre d'**ACANID** qui suivent. L'ensemble formé par la structure **ACANTB** et ses *bNId* structures **ACANID** représente ce qu'il est convenu d'appeler le *Task Block*.

Notification

La seule partie "à géométrie variable" du *Task Block* concerne le mécanisme de notification utilisé, qui dépend essentiellement du système d'exploitation sous-jacent.

Trois types de notifications ont été envisagés, qui répondent aux exigences des systèmes utilisés. Le type de notification est codé dans le champ *bNotType* :

```
/* Notification types (bNotType) */
#define TBNT_EVENT      0 /* event notification (ADTM, Win32) */
    /* ADTM_notify_event(wTid, wEvent); */
    /* SetEvent(hEvent); */
#define TBNT_CALLBACK    1 /* callback (16-bits Windows) */
    /* cbNotification(dwRefData); */
#define TBNT_POSTMESSAGE 2 /* PostMessage (16-bits Windows)*/
    /* cbNotification(hWnd, nMsg, wStatus, dwRefData) */
```

La notification par événement (*bNotType* = TBNT_EVENT) est a priori la plus efficace. Elle peut être utilisée sous MS-DOS (avec ADTM) et d'une façon générale sous Win32 (Windows 95/98, Windows NT, Windows CE). Sous MS-DOS et ADTM, une application doit mettre à jour les champs *wTid* et *wEvent* de chaque structure enregistrée. Le champ *wTid* doit être initialisé avec un identificateur de tâche retourné par ADTM suite à un

appel à **ADTM_start_task**, ou avec l'identificateur de la tâche DOS (0xFFFF); *wEvent* doit pour sa part correspondre à un masque d'événement ADTM valide. Reportez-vous à la documentation d'ADTM pour plus d'information.

Sous Win32, ACAN32.DLL se charge d'allouer un événement Win32 (*hEvent*) et un *thread* de service à chaque processus qui l'utilise. La seule chose dont doit se préoccuper l'application est de fournir dans le champ *cbNotification* du *Task Block* l'adresse d'une routine de type CALLBACK, appelée à chaque réveil du *thread* de service par l'événement *hEvent*.

La notification par événement n'existe pas sous Windows en mode 16-bits. Le mécanisme de base pour signaler un événement matériel à une application est ici l'exécution par le VxD d'une routine de CALLBACK située dans l'espace utilisateur.

Si un tel mécanisme est envisagé, l'application doit garnir le champ *bNotType* avec la valeur TBNT_CALLBACK, le champ *cbNotification* avec l'adresse de sa routine de CALLBACK, le champ *dwRefData* lui permettant de retrouver un contexte lors du rappel. La routine *cbNotification* doit impérativement résider dans de la mémoire FIXED. De plus, le programmeur doit être conscient du fait que cette routine n'a guère le droit d'appeler que **PostMessage**.

Pour simplifier le travail de notification sous Windows 16-bits, il existe une autre façon de faire, qui permet au VxD d'appeler directement **PostMessage** quand l'événement CAN se produit, avec les paramètres choisis par le programmeur.

Pour utiliser ce mécanisme privilégié, l'application doit garnir le champ *bNotType* avec la valeur TBNT_POSTMESSAGE, le champ *cbNotification* avec l'adresse de **PostMessage** et le champ *hWnd* avec le *handle* de la fenêtre chargée de traiter le message dont la teneur est indiquée dans le champ *nMsg*. Lorsque la fenêtre reçoit le message, le paramètre *wParam* de la *Windows Procedure* contient la valeur du champ *wStatus* au moment de l'appel de par le VxD et le paramètre *lParam* contient celle du champ *dwRefData*, qui permet généralement de retrouver le *Task Block*.

Quand la notification se produit, l'application se doit d'accuser réception de tous les événements qui ont pu advenir entre le moment où le driver a déclenché le mécanisme de notification et celui où elle le prend effectivement en compte. Cela doit être fait à l'aide d'une des routines de bibliothèques.

que **ACAN_AckEvent** ou **ACAN_AckEventCheck**, qui retourne le contenu du champ *wStatus* du *Task Block* tout en le remettant à 0.

```
/* Event status (LOBYTE(wStatus)) */
#define TBS_SC          0x01 /* a special condition occurred */
#define TBS_MB          0x02 /* message(s) has(have) been buffered */
#define TBS_OVERFLOW    0x04 /* message(s) has(have) been discarded */
```

La remise à 0 de *wStatus* libère le driver, qui peut alors notifier un nouvel événement. Du fait de ce fonctionnement, l'application se doit de consommer tous les événements en attente de prise en compte en bouclant jusqu'à ce que **ACAN_AckEvent** retourne 0. Si ce schéma n'est pas suivi, l'application ne doit surtout pas s'offusquer des notifications sans objet, correspondant aux événements advenus pendant le traitement des événements précédents.

Les bits de poids faibles de *wStatus* indiquent le type d'événement que l'application doit traiter :

- TBS_SC indique qu'au moins une condition spéciale filtrée par le champ *bSCFilter* est advenue. Les bits à 1 de l'octet de poids fort de *wStatus* sont alors significatifs des erreurs CAN accumulées.

```
/* Special condition bits (bSCFilter and HIBYTE(wStatus)) */
#define SC_BUSERR       0x0080 /* A Bus Error condition occurred */
#define SC_ARBLOST      0x0040 /* Arbitration Lost condition occurred */
#define SC_ERRPAS       0x0020 /* Error Passive state reached */
#define SC_WAKEUP       0x0010 /* chip has been woke-up */
#define SC_OVERRUN      0x0008 /* controller buffers overrun */
#define SC_NOBUFF       0x0004 /* internal buffers are missing */
#define SC_ERRLIM       0x0002 /* first warning before Bus Off */
#define SC_BUSOFF       0x0001 /* Bus Off state reached */
```

A chaque nouvelle occurrence d'une condition spéciale, le driver relève la valeur courante des compteurs d'erreurs en réception et en émission du contrôleur. A la notification de la condition spéciale, les dernières valeurs de ces compteurs sont recopiées dans les champs *bREC* et *bTEC* du *Task Block*. De cette façon, l'application est assurée d'avoir toujours la valeur la plus fraîche possible au moment où elle prend en compte l'événement. Les compteurs d'erreurs sont significatifs en cas d'erreur SC_BUSERR, SC_ERRPAS, SC_ERRLIM et SC_BUSOFF.

Les champs *bALC* et *bECC* sont spécifique au contrôleur utilisé. Dans le cas du SJA1000 de Philips, *bALC* est un code qui indique à quel position de bit l'arbitrage a été perdu et *bECC* un code qui indique quel type d'erreur bus est advenue. Si les conditions spéciales *SC_ARBLOST* ou *SC_BUSERR* sont inactives, la valeur de ces champs est sans signification.

Dans l'état actuel des drivers, la condition *SC_WAKEUP* ne devrait jamais se produire.

L'erreur *SC_OVERRUN* est grave et ne devrait jamais se produire dans les conditions normales. Elle est vraisemblablement l'expression d'un blocage du système qui interdit au driver de traiter les interruptions dans un temps raisonnable. Cette erreur ne doit pas être confondue avec la condition *TBS_OVERFLOW* décrite plus loin.

L'erreur *SC_NOBUFF* ne concerne pas MS-DOS. Sous Windows, elle est l'expression d'un manque de ressources internes du driver (tampons à message), détecté lors de l'interruption à la réception d'un message. Paradoxalement, ces ressources peuvent être accaparée par une surabondance de messages à émettre.

La plupart des conditions spéciales n'ont pas d'incidence véritable sur le fonctionnement des applications. Le passage dans l'état *Bus Off* est cependant totalement bloquant pour le reste de l'activité CAN. Une application pourra alors être amenée à provoquer la réinitialisation du contrôleur CAN en repassant *Bus On* à l'aide de **ACAN_BusControl**.

Il faut aussi savoir que le driver avorte de sa propre initiative la transmission des messages dont la tentative d'émission provoque une erreur bus : cela permet d'éviter d'accumuler les erreurs jusqu'à passer *Bus Off* quand la transmission ne peut vraiment pas se faire.

- *TBS_MB* indique qu'au moins un message filtré par les **ACANID** du *Task Block* a été reçu et tamponné dans la FIFO de stockage décrite par *lpRMB* et *wRMBSize*. Ces champs donnent respectivement l'adresse et la contenance en **ACANMB** du tampon de stockage intermédiaire des messages reçus. Il s'agit d'une FIFO circulaire remplie par le driver quand un message intéressant est reçu et vidée à l'initiative de l'application via

ACAN_RetrieveMessages. Sous Windows en mode 16-bits, il est indispensable que la mémoire pointée par *lpRMB* soit de type FIXED. La bibliothèque fournit les primitives d'allocation *ad hoc* : **ACAN_AllocFixedMemory** et **ACAN_FreeFixedMemory**.

- TBS_OVERFLOW est mis à 1 par le driver quand un message est reçu correctement mais qu'il est impossible de le ranger dans le tampon de l'application, faute de place. Une application qui reçoit ce genre d'événement trop systématiquement devrait envisager sérieusement d'augmenter la taille de son tampon de stockage ou, si cela est possible, de réduire significativement la fréquence d'émission des messages qu'elle doit recevoir.

Les sources des programmes de test CANTERM.EXE, WCT.EXE et WCT32.EXE illustrent parfaitement les mécanismes décrits plus haut.

Les champs *lpBuf* et *wCount* du *Task Block* sont utilisés lors de l'exécution de **ACAN_RetrieveMessages**. Tout comme les champs *lpRMB* et *wRMBSize*, ils donnent l'adresse et la taille (en nombre d'**ACANMB**) d'un tampon. Les messages récupérés par **ACAN_RetrieveMessages** sont rangés en *lpBuf*, à concurrence de *wCount* messages.

Contrairement au tampon de stockage pointé par *lpRMB*, *lpBuf* ne doit pas forcément pointer sur de la mémoire FIXED et peut être modifié à loisir entre deux appels à **ACAN_RetrieveMessages**. L'expérience prouve toutefois qu'il est à la fois simple et agréable d'utiliser toujours le même tampon pour y recevoir ses messages...

Le champ *dwFree* est libre d'utilisation.

INTERFACE DE PROGRAMMATION

La description de l'interface de programmation fait appel aux définitions suivantes :

BYTE	octet (8 bits)
WORD	mot de 16 bits
DWORD	mot de 32 bits

UINT	entier non signé (16 ou 32 bits)
PXXX	pointeur (dans le modèle en vigueur) sur un élément du type XXX
FPXXX	pointeur FAR (segment:offset) sur un élément du type XXX
LPXXX	pointeur long (32 bits) ou FAR (selon environnement) sur un élément du type XXX
ACANAPI	convention d'appel et de passage de paramètres à une fonction, variable selon système
CALLBACK	convention d'appel et de passage de paramètres à une fonction; en général, <i>pascal</i>

ACAN_Init

Initialisation de la bibliothèque.

Interface :

WORD ACANAPI ACAN_Init(void);

Retour :

0xVVRR (VV = version, RR = révision)

Remarque :

Si 0xVVRR vaut 0 (version 0.0), cela signifie que le driver n'est pas présent ou pas opérationnel.

Toute application qui utilise une bibliothèque ACAN se doit d'appeler **ACAN_Init** avant d'engager toute autre activité ACAN, sous peine de "plantages" sévères.

ACAN_GetDriverVersion

Obtention du numéro de version du driver.

Interface :

WORD ACANAPI ACAN_GetDriverVersion(void);

Retour :

0xVVRR (VV = version, RR = révision)

Remarque :

Si 0xVVRR vaut 0 (version 0.0), cela signifie que le driver n'est pas présent ou pas opérationnel.

Toute application qui utilise une bibliothèque ACAN devrait appeler **ACAN_GetDriverVersion** et vérifier que le driver est fonctionnel avant d'engager toute autre activité ACAN, sous peine de "plantages" sévères.

ACAN_GetInitStatus

Obtention du type de contrôleur CAN utilisé et de son état d'initialisation.

Interface :

WORD ACANAPI ACAN_GetInitStatus(void);

Retour :

0xSSTT :

SS = état des initialisations matérielles

0 = matériel non initialisé

X = matériel initialisé

TT = type de contrôleur utilisé

0 = carte CAN PC/104

Remarque :

Dans leur état actuel, les programmes de test ne supportent que la carte CAN PC/104 d'ATON systèmes et son contrôleur Philips SJA1000 (TT = 0). Les applications peuvent vérifier que TT est bien à 0 et refuser d'aller plus loin si elles ne sentent pas capables de continuer avec un contrôleur inconnu.

ACAN_SetHardConfig

Initialisations matérielles du contrôleur.

Interface :

BOOL ACANAPI ACAN_SetHardConfig(LPACANHI lpHI);

Entrée :

lpHI pointeur sur la structure **ACANHI** décrivant les initialisations à effectuer

Retour :

TRUE si les initialisations ont réussi, FALSE sinon

Remarque :

En cas d'échec, la première chose à faire est de vérifier attentivement la composition de la structure **ACANHI** passée à la fonction :

le passage d'un paramètre invalide est à 99% la cause probable de cet échec.

En cas de succès, plusieurs faits sont à prendre en compte :

- Les numéros de machine (MID) et *broadcast* (BID) sont mis à jour tant sur le plan physique que sur le plan logique, ce qui peut avoir des conséquences importantes en cas de réinitialisation : les masques de filtrage du contrôleur sont forcés avec les valeurs proposées et les prochains filtres logiciels enregistrés hériteront du nouveau MID. Si le nouveau filtrage est plus restrictif que l'ancien, les anciens filtres ne pourront plus fonctionner et les applications qui les ont installées ne verront plus rien passer...
- Le contrôleur est déconnecté du bus CAN (*bus off*) : un appel à **ACAN_BusControl(BUS_ON)** doit suivre pour que l'utilisation du bus devienne effective.

ACAN_GetHardConfig

Récupération de la configuration matérielle du contrôleur.

Interface :

BOOL ACANAPI ACAN_GetHardConfig(LPACANHI lpHI);

Entrée :

lpHI pointeur sur la structure ACANHI destinée à recevoir la description des initialisations effectuées

Retour :

TRUE si la structure ACANHI a été remplie, FALSE sinon

Remarque :

Le fait que la fonction retourne FALSE indique essentiellement l'état de non-initialisation du matériel.

La récupération de la configuration matérielle existante peut permettre aux applications qui doivent obligatoirement forcer leur propre configuration de le faire avec le maximum de douceur. Par exemple, une application qui doit changer la structure du numéro de machine peut le faire sans reprogrammer la vitesse de fonctionnement du bus, tout comme une application qui devrait s'adapter à un changement de vitesse du bus pourrait le faire sans modifier les masques de filtrage existants. Il faut tout de même être conscient du fait qu'une application qui reprogramme la configuration

matérielle va laisser le contrôleur déconnecté du bus pendant un certain temps, pendant lequel tous les messages qui auraient pu y transiter seront perdus.

ACAN_BusControl

Connecte ou déconnecte le contrôleur CAN du bus.

Interface :

```
typedef enum {  
    BUS_OFF = 0,  
    BUS_ON = 1  
} BUS_STATE;
```

WORD ACANAPI ACAN_BusControl(BUS_STATE bsNewState);

Entrée :

bsNewState état désiré (*Bus On/Off*)

Retour :

0xOONN :

OO état précédent du bus

NN nouvel état du bus

Au sortir de **ACAN_SetHardConfig**, le contrôleur est déconnecté du bus. Cela permet aux applications qui initialisent le matériel de conduire leurs propres initialisations et d'enregistrer tous les *Task Blocks* dont elles ont besoin en toute sérénité, sans craindre d'être dérangées par l'arrivée de messages ou l'occurrence d'erreurs.

En contrepartie, toute application appelant **ACAN_SetHardConfig** se doit d'appeler aussi **ACAN_BusControl(BUS_ON)** pour autoriser l'activité CAN : quand le contrôleur est dans l'état *Bus Off*, aucune activité CAN n'est possible pour les applications locale et tous les messages qui transitent sur le bus et destinés à ces applications sont irrémédiablement perdus.

Si le taux d'erreurs liées à la réception ou à l'émission de messages sur le bus CAN devient trop important, le contrôleur CAN se déconnecte automatiquement du bus, conformément aux recommandations de la norme. Le passage du contrôleur dans l'état *Bus Off* fait l'objet d'une notification de Condition Spéciale aux applications, qui peuvent alors prendre les mesures qui s'imposent, comme par exemple repasser dans l'état *Bus On*.

Pour gérer efficacement ce comportement, les drivers prennent deux mesures importantes :

- Le passage de l'état Bus Off à l'état Bus On provoque la remise à 0 des compteurs d'erreur du contrôleur et l'avortement de toute émission en attente : le contrôleur se retrouve alors au repos et prêt à fonctionner à nouveau dans les meilleures conditions possibles, pour peu que l'état de santé général du bus s'y prête.
- Le mode de fonctionnement spontané du contrôleur CAN est tel qu'il essayera de transmettre un message que le driver lui a soumis par tous les moyens, c'est-à-dire en cumulant éventuellement les erreurs de bus jusqu'à passer dans l'état *Bus Off*. Ce sera par exemple le cas si un message est envoyé sur le bus sans qu'aucun interlocuteur ne le prenne en charge, ou si les vitesses de fonctionnement du contrôleur et du bus diffèrent. Pour éviter cette accumulation d'erreurs et cette déconnexion du bus qui dans bien des cas peut être considérée comme excessive, le driver avorte impitoyablement l'émission d'un tel message au moment où il détecte l'erreur de bus CAN. Noter que l'erreur SC_BUSERR est de toutes façons signalée aux applications, qui pourront prendre les mesures qui leur semblent nécessaires.

ACAN_Register

Enregistrement d'un *Task Block* auprès du driver.

Interface :

BOOL ACANAPI ACAN_Register(LPACANTB lpTB);

Entrée :

lpTB adresse d'une structure **ACANTB** pré-initialisée

Retour :

TRUE si la structure a été enregistrée, FALSE sinon.

Remarque :

Dans les conditions normales, l'enregistrement ne peut échouer que dans la mesure où les champs du *Task Block* n'ont pas correctement été initialisés. En deuxième approche, il est possible que les drivers fonctionnant sous Windows remontent à l'application des erreurs liées à l'épuisement de certaines ressources du système (comme par exemple les entrées dans les tables de pagination); de telles erreurs

sont cependant extrêmement improbables et sont vraisemblablement le signe d'un dysfonctionnement plus grave.

Le *Task Block* est l'ensemble constitué par la structure **ACANTB** décrite précédemment, par les éventuelles structures **ACANID** qui la suivent directement, et, dans la plupart des cas par le tampon FIFO servant au driver pour y ranger les messages destinés à ce *Task Block*. Bien que l'adresse de ce tampon bénéficie d'un champ particulier dans la structure **ACANTB** (*lpRMB*), il est recommandé de l'allouer à la suite des **ACANID**, de façon à former un bloc homogène. Cela ne dispense pas d'initialiser correctement le champ *lpRMB* !

Quel que soit le système d'exploitation sous-jacent, l'ensemble du *Task Block* est accédé par le driver lors du traitement des interruptions du contrôleur CAN. Cela contraint le *Task Block* à une certaine fixité, le *swap* ou le déplacement inopiné du contenu de sa mémoire étant tout sauf bienvenu. Sous les systèmes comme Windows où la mémoire est sujette à divers mouvements du fait de sa "virtualisation", la constitution d'un bloc homogène simplifie le travail du programmeur, qui n'a besoin d'allouer qu'un seul bloc de mémoire de type **FIXED** pour tous ses besoins.

Les fonctions **ACAN_AllocFixedMemory** et **ACAN_FreeFixedMemory** décrites plus loin permettent aux applications d'allouer et de libérer facilement la mémoire dont ils ont besoin pour le *Task Block*.

Le gros du travail du programmeur consiste à initialiser les champs de la structure, ce qui peut prendre une tournure différente en fonction du système d'exploitation utilisé.

Sous MS-DOS (ACAN.LIB) :

<i>bNotType</i>	TBNT_EVENT (0)
<i>wTid</i>	TID (<i>Task ID</i>) ADTM de la tâche à qui signaler l'événement <i>wEvent</i> .
<i>wEvent</i>	Code ADTM de l'événement susnommé.

L'application démarre généralement via **ADTM_start_task** une tâche chargée de la prise en compte et du traitement des événements CAN. Cette tâche dort en attendant l'événement *wEvent* (**ADTM_wait_event**), signalé par le driver à la tâche *wTid* lors de l'interruption (**ADTM_notify_event**).

Sous Windows 16 bits (ACAN.DLL) :

bNotType TBNT_POSTMESSAGE (2)
hWnd *handle* de la fenêtre à qui **PostMessage** enverra le message de notification *nMsg*.
nMsg Code du message susnommé.
cbNotification Adresse de **PostMessage**.
dwRefData Données passées dans *dwParam* de **PostMessage**.

C'est le mode de fonctionnement privilégié. Les événements CAN sont répercutés à l'application sous forme de messages Windows et sont par conséquent traités dans le cadre d'une *Window Procedure*, ce qui ne devrait pas décontenancer le développeur habitué à cet environnement de programmation. Noter que bien que le champ *cbNotification* soit généralement initialisé avec l'adresse de **PostMessage**, l'adresse d'une routine dont l'interface est équivalente peut aussi être fournie. Si une routine autre que **PostMessage** est utilisée, il faut cependant rester conscient des restrictions de contexte liées aux routines de CALLBACK directement invoquées par le noyau, qui n'ont guère que le droit d'appeler **PostMessage** !

ou bien :

bNotType TBNT_CALLBACK (1)
cbNotification Adresse de la routine de notification.
dwRefData Paramètre unique de la routine de notification.

Le driver appelle la routine *cbNotification* (de type CALLBACK) avec le paramètre *dwRefData*. La routine de notification n'a droit qu'aux appels de type **PostMessage** ou **MessageBeep**.

Sous Windows 32 bits (ACAN32.DLL) :

cbNotification Adresse de la routine de notification
dwRefData paramètre éventuel de la routine de notification

La DLL force le champ *bNotType* à TBNT_EVENT (0) et assure elle-même l'initialisation du champ *hEvent* avec le *handle* d'un événement Win32 converti pour pouvoir être utilisé par le VxD. En fait, pour chaque *process* Win32 utilisateur de la DLL, un de ces

événements est créé et un *thread* attendant que cet événement soit signalé est lancé. Le *thread* est réveillé par le VxD quand un événement CAN se produit. Il se charge alors d'appeler la routine *cbNotification* de chaque *Task Block* enregistré par le *process*, avec pour paramètre l'adresse de ce même *Task Block*. La routine de notification (de type CALLBACK) est alors libre d'exploiter le champ *dwRefData* de la structure comme bon lui semble.

Sous tous les systèmes :

<i>bSCFilter</i>	Filtre des conditions spéciales pour lesquelles l'application souhaite être notifiée.
<i>lpRMB</i>	Adresse du tampon de stockage des messages destinés à ce <i>Task Block</i> .
<i>wRMBSize</i>	Capacité du tampon de stockage, en messages (ACANMB).
<i>bNId</i>	Nombre de filtres (ACANID) qui suivent la structure ACANTB .
<i>dwAppUsage</i>	32 bits utilisables par l'application.

Une application est notifiée quand au moins l'une des conditions spéciales à laquelle elle s'est déclarée sensible se produit. Cela est visible dans le bit TBS_SC du champ *wStatus*, récupéré via **ACAN_AckEvent** ou **ACAN_AckEventCheck**. L'application doit alors consulter les bits de poids fort de *wStatus* pour prendre connaissance de la teneur exacte des conditions spéciales advenues depuis la notification précédente.

Quand une application enregistre les filtres idoines et que *bNId* est différent de 0 (il y a un tampon pour ranger les messages reçus), elle est aussi notifiée de la réception de messages CAN. Tout comme pour les conditions spéciales, cela est visible dans le bit TBS_MB du champ *wStatus*. L'application doit alors récupérer et examiner les messages reçus depuis la notification précédente, à l'aide de **ACAN_RetrieveMessages**.

ACAN_AckEvent, ACAN_AckEventCheck
--

Accuse réception de tous les événements en attente de prise en compte pour un *Task Block*.

Interface :

WORD ACANAPI ACAN_AckEvent(LPACANTB lpTB);
WORD ACANAPI ACAN_AckEventCheck(LPACANTB lpTB);

Entrée :

lpTB adresse d'une structure **ACANTB** enregistrée

Retour :

valeur de *wStatus* au moment de l'appel (événements et conditions spéciales actives)

Remarque :

Les champs dont les valeurs sont retournées sont remises à 0 par l'appel. L'utilisation de **ACAN_AckEvent** est plus rapide que celle de **ACAN_AckEventCheck** mais ne vérifie pas la validité du pointeur *lpTB* passé en paramètre.

Les applications doivent accuser réception des événements qui leur sont notifiés pour que de nouvelles notifications puissent se produire. En raison de la façon de procéder recommandée, il est possible que des notifications sans objet se produisent. Le traitement d'une notification devrait consister à :

1. Récupérer les événements en attente par un appel à **ACAN_AckEvent** ou à **ACAN_AckEventCheck**.
2. Traiter les débordements si le bit **TBS_OVERFLOW** est à 1.
3. Traiter les conditions spéciales si le bit **TBS_SC** est positionné.
4. Si le bit **TBS_MB** est à 1, récupérer tous les messages reçus et stockés en invoquant **ACAN_RetrieveMessages**, jusqu'à ce que plus aucun message ne soit disponible.

Comme on peut le remarquer, il est possible que de nouveaux messages soient reçus (ou que de nouvelles conditions spéciales se déclarent) après l'accusé de réception de l'étape 1 et avant la fin de l'étape 4 : un nouvel événement sera préparé par le driver mais la boucle de l'étape 4 purgera tous les messages en attente, y compris les nouveaux. Le résultat sera une nouvelle notification sans objet.

Si l'activité CAN est très soutenue, il est conseillé pour éviter cela de boucler sur l'étape 1 jusqu'à ce qu'il n'y ait plus d'événements à traiter.

ACAN_Deregister

Désenregistrement d'un *Task Block* auprès du driver.

Interface :

BOOL ACANAPI ACAN_Deregister(LPACANTB lpTB);

Entrée :

lpTB adresse d'une structure **ACANTB** enregistrée

Retour :

TRUE si la structure a été libérée (*lpTB* valide), FALSE sinon.

Un *Task Block* enregistré auprès du driver par une application doit impérativement être désenregistré avant que l'application ne se termine, sous peine de "plantages" violents !

ACAN_SendMessage

Envoi d'un message CAN.

Interface :

**BOOL ACANAPI ACAN_SendMessage(LPACANMB lpMsg,
BYTE bPrio);**

Entrée :

lpMsg adresse du message (**ACANMB**) à émettre
bPrio priorité d'émission (0 = forte, 255 = faible)

Retour :

TRUE si le message a été mis en queue ou envoyé, FALSE sinon.

Si possible, le message est immédiatement transmis sur le bus CAN. Sinon, il est recopié dans un tampon intermédiaire et rangé dans la file des messages à émettre, en fonction de sa priorité.

La fonction ne retourne FALSE que dans la mesure où il ne reste plus de tampon interne libre pour y recopier le message. L'application doit alors attendre un certain temps avant de tenter à nouveau l'émission. Si tout se passe bien, les messages sont émis à peu de choses près par le driver à la vitesse du bus CAN (les messages mis en queue sont émis sur interruption, ce qui rajoute un petit délai, dépendant de la machine et du système utilisés).

Les messages dotés de la priorité 0 bénéficient d'un traitement particulier dans la mesure où ils sont émis sur le bus et "reçus" localement. Cela peut

être utilisé pour certains tests, pour des applications ayant un besoin de multi-diffusion totale, voire comme une certaine forme de communication inter-processus.

ACAN_RetrieveMessages

Récupération des messages CAN reçus et stockés dans un *Task Block*.

Interface :

int ACANAPI ACAN_RetrieveMessages(LPACANTB lpTB);

Entrée :

lpTB adresse d'une structure **ACANTB** enregistrée

Retour :

Nombre de messages récupérés, ou -1 en cas de problème.

Remarque :

La fonction retourne la valeur -1 en cas de problème. Cela peut correspondre au passage d'un *lpTB* invalide mais aussi à une valeur illégale du pointeur *lpBuf*.

Les messages reçus pour un *Task Block* sont stockés temporairement par le driver dans le tampon FIFO repéré par *lpRMB* et *wRMBSize*. Un autre tampon doit être utilisé par l'application pour que le driver puisse y vider les messages préalablement tamponnés, à la demande de l'application. Bien que pouvant paraître un peu lourd, ce mécanisme permet de s'affranchir des problèmes d'exclusion mutuelle qui peuvent exister entre une routine de traitement d'interruption en mode noyau et une application évoluant dans les hautes sphères.

Contrairement au tampon de stockage intermédiaire, le tampon de l'application peut être situé dans de la mémoire ordinaire. Il est repéré par les champs *lpBuf* et *wCount* de l'**ACANTB**.

Un seul appel à **ACAN_RetrieveMessages** permet de récupérer jusqu'à *wCount* messages (**ACANMB**) en *lpBuf*, rangés par ordre chronologique.

ACAN_AllocFixedMemory

Allocation de mémoire inamovible (FIXED).

Interface :

LPVOID ACANAPI ACAN_AllocFixedMemory(UINT nBytes);

Entrée :

nBytes nombre d'octets à allouer

Retour :

Adresse du bloc de mémoire alloué ou NULL en cas de problème.

Cette fonction permet de masquer aux applications les différences existant entre les différents systèmes en ce qui concerne l'allocation de mémoire, et la possibilité de "verrouillage" de cette mémoire.

Sous MS-DOS, où la mémoire est bien calme, cette fonction fait une allocation ordinaire en utilisant la fonction "Allocate Memory" du système (INT 21h, AH = 48h).

Sous Windows 3, la mémoire est allouée via **GlobalAlloc**, en spécifiant l'indicateur GMEM_FIXED (en fait, GPTR).

Sous Win32, le driver se chargeant de verrouiller la mémoire dont il a besoin, **ACAN_AllocFixedMemory** se contente d'allouer ce qu'on lui demande en utilisant **VirtualAlloc** et **VirtualLock**.

ACAN_FreeFixedMemory

Libère la mémoire allouée par **ACAN_AllocFixedMemory**.

Interface :

**BOOL ACANAPI ACAN_FreeFixedMemory(LPVOID lpAlloc,
 UINT nBytes);**

Entrée :

lpAlloc adresse du bloc précédemment alloué

nBytes taille du bloc précédemment alloué

Retour :

TRUE si tout c'est bien passé, FALSE en cas de problème.

La mémoire allouée via **ACAN_AllocFixedMemory** doit être libérée par **ACAN_FreeFixedMemory**.

ACAN_Exit

Sortie propre d'une application.

Interface :

void ACANAPI ACAN_Exit(void);

Toute application ayant utilisé les services de la bibliothèque ACAN ou ACAN32 devrait appeler cette fonction avant de se terminer.

BIBLIOTHEQUE C/C++

MS-DOS

L'interface de programmation native du driver **ACAN.EXE** est conçue pour être utilisée à partir de l'assembleur x86. Toutefois, pour permettre au plus grand nombre d'utiliser ce driver, une bibliothèque d'interface au C est fournie.

Cette bibliothèque consiste en un fichier d'en-tête à inclure dans les sources des applications utilisant ACAN ainsi qu'en deux fichiers « .LIB » contenant le code assurant l'interface entre l'application et le driver pour les deux grandes familles de modèles C. Ces fichiers sont présents dans le répertoire DEV de la disquette d'installation.

ACAN.H	définitions pour l'utilisation du driver
ACANN.LIB	bibliothèque pour les modèles <i>near</i> du C (<i>Tiny, Small, Compact</i>)
ACANF.LIB	bibliothèque pour les modèles <i>far</i> du C (<i>Medium, Large, Huge</i>)

En fonction de ses besoins, une application utilisera l'une ou l'autre version de la bibliothèque.

Windows 3.1x

Pour simplifier l'utilisation du VxD **VACAND.386** par les applications, la bibliothèque d'édition de liens dynamique ACAN.DLL est fournie. Le fichier de définitions ACAN.H et le *stub* statique ACAN.LIB qui permettent aux programmeurs d'utiliser la DLL sont aussi fournis. Ces fichiers sont présents dans le répertoire DEV de la disquette d'installation.

ACAN.H	définitions pour l'utilisation du driver
ACAN.DLL	bibliothèque à édition de liens dynamique assurant l'interface entre les applications et le VxD
ACAN.LIB	bibliothèque permettant l'utilisation de ACAN.DLL par les programmes

Windows 95

Pour simplifier l'utilisation de **VACAND.VXD** par les applications, les bibliothèques d'édition de liens dynamique **ACAN.DLL** et **ACAN32.DLL** sont fournies. Le fichier de définitions **ACAN.H** et les *stubs* statiques **ACAN.LIB** et **ACN32.LIB** qui permettent aux programmeurs d'utiliser la DLL sont aussi fournis.

ACAN.H	définitions pour l'utilisation du driver
ACAN.DLL	bibliothèque à édition de liens dynamique assurant l'interface entre les applications 16-bits et le VxD
ACAN32.DLL	bibliothèque à édition de liens dynamique assurant l'interface entre les applications 32-bits et le VxD
ACAN.LIB	bibliothèque permettant l'utilisation de ACAN.DLL par les programmes
ACAN32.LIB	bibliothèque permettant l'utilisation de ACAN32.DLL par les programmes

ANNEXE A : Interface de programmation assembleur

Sous MS-DOS, il est possible d'utiliser les services du driver ACAN.EXE à partir de programmes ou de routines écrits en assembleur. Le fichier ACAN.INC fourni avec l'environnement de développement contient les définitions nécessaires à cette utilisation.

Test de présence du driver ACAN

La détection de la présence du driver ACAN est réalisée suivant un processus standard (*Alternate Multiplex Interrupt Specification* ou AMIS) utilisant l'interruption 2Dh. Contrairement à beaucoup de TSR utilisant l'interruption 2Fh et un numéro fixe (*multiplex number*) dans AH, le driver ACAN utilise un numéro variable qui est déterminé par la méthode suivante:

Si vecteur d'interruption 2Dh (DWORD à l'adresse 0000:00B4) est nul

Fin: le driver ACAN n'est pas installé

AH = 00

Boucle:

AL = 00

Appel INT 2D

Si AL = FF

Si la chaîne pointée par DX:BX est "**Aton SysACAN**" (16 octets)

Fin: le numéro est dans AH

AH++

Si AH! = 0 va en 'Boucle'

Fin: le driver ACAN n'est pas installé

Si le numéro est trouvée, les fonctions suivantes sont accessibles à travers l'interruption 2Dh:

Int 2D, AH = numéro, AL = 00 - Test présence driver (utilisée précédemment)

Retour:AL = FF

CX = 0100 (version 1, révision 0)


```

DX:BX-> db "Aton Sys"      ; constructeur (8 caractères)
          db "ACAN"         " ; produit (8 caractères)
          db "ATON Controller Area Network Driver"
                                   ;description
          db 0

```

Int 2D, AH = numéro, AL = 01 - Demande point d'entrée driver

Retour:AL = FF

DX:BX = point d'entrée driver ACAN

Int 2D, AH = numéro, AL = 02 - Désinstallation driver

Retour:AL = 01 si driver toujours en place

AL = 04 si driver désactivé

BX = segment bloc mémoire à libérer

Int 2D, AH = numéro, AL = 04 - Demande liste des interruptions détournées

Retour:AL = 04

DX:BX-> db numéro interruption

dw offset point d'entrée (segment = DX liste)

...

db 2D

La fonction AL = 1 doit être appelée pour obtenir un pointeur sur le point d'entrée du driver ACAN. L'interruption 2Dh n'est utilisée que pour détecter la présence du noyau et obtenir le point d'entrée du noyau. Elle ne permet pas d'accéder aux services.

Appel des services du driver

Le pointeur retourné durant la phase de détection de présence du driver ACAN doit être sauvé dans une variable statique. Un appel à un service du driver est réalisé en chargeant les registres avec les paramètres d'entrée et en exécutant un CALLF sur le pointeur. Pour respecter l'esprit du C, les valeurs sont retournées en AX et DX et les registres "sensibles" du C (DI, SI, BP, SP, DS, SS) sont préservés. Pour faciliter le travail de l'assembleur, les fonctions de service préservent aussi BX et ES.

Le code du service est passé dans AL. En général, les pointeurs sur structures sont passés en ES:BX et les erreurs sont signalées par le retour de la valeur 0 (FALSE) en AX.

Les services d'ACAN.EXE sont exploités directement par les bibliothèques ACANN.LIB et ACANF.LIB, dont l'interface de programmation C qu'elles offrent a déjà été amplement décrite. La plupart des services ont une correspondance directe avec une fonction **ACAN_Xxx** de ACANX.LIB fonctionnellement équivalente. Les structures utilisées sont les mêmes.

ACAN_GET_VERSION et ACAN_GET_VERSION_1 (récupération du numéro de version du driver)

Entrée: AL = 0 ou 9

Retour: AH = numéro de version (major)
AL = numéro de révision (minor)
DL = type de contrôleur (CANC_SJA1000)
DH = état des initialisations du matériel
(0 = pas initialisé, 1 = initialisé)

La présence de deux codes pour le même service est due à un comportement désagréable de Windows 95 qui refuse aux applications le droit d'appeler les VxD avec le code IOCTL 0. Pour ceux que cela choque de voir que ce qui se passe sous Windows 95 peut avoir une influence sous MS-DOS, nous rappelons que de gros efforts ont été fait pour présenter une interface unifiée entre ces différents systèmes. En pratique, le même fichier ACAN.INC est utilisé pour la génération de ACAN.EXE, VACAND.386 et VACAND.VXD, à quelques *define* près.

Le service ACAN_GET_VERSION_1 est appelé par les fonctions **ACAN_GetDriverVersion** et **ACAN_GetInitStatus** des bibliothèques.

ACAN_HARD_CONFIGURE (initialisation du matériel)

Entrée: AL = 1
ES:BX = adresse structure **his** (lpHI)

Retour: AX = 0 si problème
AX = 1 si OK

La structure **his** dont l'adresse est passée en paramètre est la version assembleur de la structure C **ACANH1**.

Ce service est appelé par la fonction **ACAN_SetHardConfig** des bibliothèques.

ACAN_HARD_STATUS (état des initialisations du matériel)

Entrée: AL = 2
ES:BX = adresse structure **his** à garnir (lpHI)

Retour: AX = état des initialisations (0 = vierge, 1 = initialisé)
ES:BX = **his** à jour si AX = 1
DL = type de contrôleur (CANC_SJA1000)

La structure **his** dont l'adresse est passée en paramètre est la version assembleur de la structure C **ACANH1**.

Ce service est appelé par la fonction **ACAN_GetHardConfig** des bibliothèques.

ACAN_HARD_CONTROL (contrôle de la présence sur le bus CAN)

Entrée: AL = 3
DL = état souhaité (0 = Bus Off, 1 = Bus On)

Retour: AL = état obtenu
AH = état précédent

Ce service est appelé par la fonction **ACAN_BusControl** des bibliothèques.

ACAN_REGISTER (enregistrement d'un *Task Block*)

Entrée: AL = 4
ES:BX = adresse structure **tbs** à enregistrer (lpTB)

Retour: AX = 1 si la structure a été enregistrée, 0 sinon.

La structure **tbs** dont l'adresse est passée en paramètre est la version assembleur de la structure C **ACANTB**.

Ce service est appelé par la fonction **ACAN_Register** des bibliothèques.

ACAN_ACK (accusé de réception d'événement sur un *Task Block*)

Entrée: AL = 5

ES:BX = adresse du *Task Block* (lpTB)

Retour: AL = ES:[BX].tb_sta

AH = ES:[BX].tb_sca

soit : AX = ES:[BX].tb_wsta (*lpTB->wStatus*)

La structure **tbs** dont l'adresse est passée en paramètre est la version assembleur de la structure C **ACANTB**.

Ce service est appelé par la fonction **ACAN_AckEventCheck** des bibliothèques.

ACAN_FREE (libération d'un *Task Block*)

Entrée: AL = 6

ES:BX = adresse du *Task Block* à libérer (lpTB)

Retour: AX = 1 si la structure a été désenregistrée, 0 sinon.

La structure **tbs** dont l'adresse est passée en paramètre est la version assembleur de la structure C **ACANTB**.

Ce service est appelé par la fonction **ACAN_Deregister** des bibliothèques.

ACAN_SEND (envoi d'un message)

Entrée: AL = 7

ES:BX = adresse structure **amb**s à garnir (lpMsg)

CL = priorité d'émission (0 = forte, 255 = faible)

Retour: AX = 1 si le message a été mis en queue ou envoyé, 0 sinon.

La structure **ambs** dont l'adresse est passée en paramètre est la version assembleur de la structure C **ACANMB**.

Ce service est appelé par la fonction **ACAN_SendMessage** des bibliothèques.

ACAN_RECEIVE (récupération de messages)

Entrée : AL = 8

ES:BX = adresse du *Task Block* de travail (lpTB)

Retour: AX = nombre de messages récupérés, ou -1 en cas de problème

La structure **tbs** dont l'adresse est passée en paramètre est la version assembleur de la structure C **ACANTB**.

Ce service est appelé par la fonction **ACAN_RetrieveMessages** des bibliothèques.

FICHE DE RAPPORT D'INCIDENT

Avant de retourner le produit, il est nécessaire de prendre contact avec le Service Après-Vente d'ATON Systèmes afin d'obtenir un numéro de retour (RMA) à inscrire dans le cadre de ce document. Les informations qui seront indiquées sur la présente fiche permettront de déterminer la(es) cause(s) de panne ou de dysfonctionnement et ainsi d'améliorer la qualité de nos services. Merci de compléter et de retourner cette carte.

Société :	Date : / /
Adresse :	Retourner à : ATON Systèmes 14 rue Charles Martigny 94700 Maisons Alfort - France
Téléphone : ()	Tél. : 0142071800
Nom :	Fax : 0142078555

Référence Produit : SBC445	
Version ROM :	N° de Série :

Processeur :	Disque dur :	Mémoire :
Vitesse :	Lecteur Souple :	
Carte d'affichage :	Autres cartes contrôleur :	Autres (réseaux ...) :
Système d'exploitation :	Version :	Langue :
Logiciel d'application :	Version :	Langue :

Messages d'erreur par le système d'exploitation :
Autres :
Avez-vous déjà rencontré ce problème ? OUI / NON
Fréquence :
Indiquer les opérations provoquant l'apparition de l'anomalie

Solution RMA <i>Cadre Réserve</i>	
--	--